Design and Implementation of a Tool for the Analysis and Exploration of Trace Graphs

Master Thesis

presented by
Felix Krömer
Matriculation Number 1634604

submitted to the
Institute for Enterprise Systems
Dr. Christian Bartelt
University of Mannheim

April 2024

Abstract

Self-driving vehicles are expected to become an integral part of transportation in the future. Due to the serious consequences that malfunctions of these autonomous systems can have on human well-being, comprehensive testing is more important than in almost any other AI application. In this context, it is essential to consider not only situations that commonly occur during driving but also to identify edge cases that tend to cause the most damage. Traditional approaches to generating and specifying test scenarios tend to miss these outliers. Many approaches either rely on expert knowledge and are therefore prone to human bias, or are based on the analysis of traffic data but rely on rules and structures that are too rigid to effectively identify edge cases. Metzger et al. introduce a novel approach for identifying edge cases that is data-driven and robust to human biases. In their approach, they transform the large amounts of sensor data recorded during test drives into a condensed graph, called a Trace Graph. Using this graph, test drives can be compared to each other, and edge cases can be efficiently identified. The goal of this thesis is to design and implement a tool for visualizing, exploring, and analyzing Trace Graphs. It allows the user to gain a deep understanding of the graph structure while remaining easy to use. At the same time, it is efficient and operates with minimal memory overhead by using algorithms specialized for Trace Graphs. This is demonstrated by tests that measure the tool's performance under load.

Contents

1 Introduction **1**

2 Fundamentals **3**
2.1 Scenarios and Scenes 3
2.1.1 Test Case Generation 4
2.1.2 Scenario Based Testing 5
2.2 Trace Graphs 6
2.2.1 Traces 6
2.2.2 Discretization 6
2.2.3 Graph Construction 8
2.2.4 Graph Comparison 9
2.2.5 PDM Optimization 10
2.3 OSGi 11
2.3.1 Module Layer 12
2.3.2 Life Cycle Layer 12
2.3.3 Service Layer 13
2.4 Cytoscape 13
2.4.1 Architecture 14
2.4.2 Tasks 15
2.4.3 Events 16
2.4.4 Data Model 16
2.4.5 Visual Styles 19

3 Contribution **21**
3.1 Requirements 21
3.2 General Architecture and Design 22
3.2.1 Points of User Interaction 22
3.2.2 Model 24

3.2.3 Controller 27
3.2.4 View 30
3.3 Exploration & Manipulation 30
3.3.1 Trace Graph Import 30
3.3.2 General Workflow and Layout 34
3.3.3 Merging and Splitting 35
3.3.4 Style and Mappings 38
3.3.5 Bin Modifications 40
3.3.6 Hiding Nodes 41
3.3.7 Update Procedure 42
3.4 Analysis 42
3.4.1 Rendering Modes 42
3.4.2 Comparing Nodes and Finding the Minimal Subtrace . . . 43
3.4.3 Comparison 44
3.5 Algorithms 44
3.5.1 Loading Trace Graphs 44
3.5.2 Updating Trace Graphs 47
3.5.3 Finding the Minimal Subtrace 51
3.5.4 Minimal Subtrace using ILP 53
3.6 Misc 55
3.6.1 Ding Renderer Improvements 55
3.6.2 Custom CyTable 55

4 Evaluation 57
4.1 Methodology 57
4.2 Individual Performance Assessment 58
4.2.1 Loading Traces 58
4.2.2 Updating Trace Graphs 59
4.2.3 Memory Usage 61
4.2.4 Identification of Memory Leaks 65
4.3 Relative Performance Evaluation 66
4.3.1 Trace Graph Updates 67
4.3.2 Custom CyTable Implementation 67
4.4 Discussion 68

5 Future Work 70

6 Conclusion 72

Chapter 1

Introduction

In recent years, there has been significant advancement in automated driving systems (ADS). Features such as adaptive cruise control and lane-keeping assistance have already become well-established. However, systems of level 3 and higher, which can handle most aspects of driving autonomously under certain conditions, are still in the testing phase and have not yet achieved widespread use [6]. Demonstrating the safety and reliability of these systems presents a significant challenge. Unlike simpler systems that need to manage a more limited number of scenarios, ADS of level 3 and higher must be capable of handling an almost infinite range of driving situations, making testing exponentially more complex [3]. Additionally, an ADS of level 3 or higher requires full autonomy under specific conditions, unlike levels 1 and 2, which require constant human oversight, thereby increasing safety and testing requirements.

Automated driving systems are primarily tested through two key approaches: scenario-based testing and generic testing. Scenario-based testing is concerned with assessing the security of an ADS on a set of predefined, challenging scenarios. This approach aims to verify that the automated driving system can handle a wide range of scenarios effectively and make appropriate decisions [16]. It stands in contrast to generic testing in regular traffic, which tries to demonstrate that a vehicle can reach a certain mileage without significant issues. However, here, the ADS will predominantly encounter simpler, more common scenarios that offer limited insight into its overall performance. Rare scenarios, which may lead to accidents, occur only every few million kilometers [11]. However, it is these scenarios that ultimately determine how secure the system really is and define the level of trust that can be placed in it. Scenario-based testing seeks to isolate and test these critical

scenarios to assess the automated driving system's reliability and safety.

The testing is most commonly performed in a virtual environment using high-fidelity simulators. As an alternative, it is also possible to test scenarios on a closed-off track using real hardware and by utilizing dummies for the other traffic participants [9]. However, moving the testing from the real world to a simulation has multiple benefits. It is significantly cheaper, saves time, and allows for the testing of scenarios that are hard to reproduce in real life. On the other hand, even the most detailed simulators cannot match the complexity of the real world, and simulated testing may therefore not be fully representative [23, 7].

Techniques for the identification of test scenarios can be broadly classified as knowledge-based, which relies on expert knowledge to identify scenarios, and data-driven, which analyzes traffic data, primarily in the form of recorded test drives, for scenarios [16]. Trace Graphs, as introduced by Metzger et al. [15], allow for the identification of scenarios by transforming the data that is collected during one or multiple test drives into a condensed graph. Paths in this graph represent sequences of abstract situations that the vehicle encountered during the test drives. Therefore, paths resemble possible scenarios.

This thesis describes the design and implementation of an application for the visualization of Trace Graphs. It is built on top of the Cytoscape platform, which is a software system for the visualization of networks. The application that is presented in this thesis allows the user to load, compare, modify, and analyze Trace Graphs in a variety of ways. In order to maximize performance and offer an uncomplicated user experience, it is tightly coupled with Cytoscape and is tailored towards its data format. Chapter 2 will explain Scenario Based Testing, Trace Graphs, OSGi (Open Service Gateway Initiative), and Cytoscape. Chapter 3 will cover the features that the application provides as well as details about their implementation, and the algorithms that were designed to enable them. Afterwards, the application will be evaluated based on an appropriate usage scenario. The final chapter provides a conclusion about the work.

Chapter 2

Fundamentals

2.1 Scenarios and Scenes

Ulbrich et al. describe a scenario as a "temporal development between several scenes in a sequence of scenes" and a scene as "a snapshot of the environment including the scenery and dynamic elements, as well as all actors' and observers' self-representations, and the relationships among those entities" [21]. Thus, a scene does not only contain static information such as the road layout, topology, traffic signs, trees, etc. but also the actors' individual, subjective perceptions of their surroundings and their relations. A scenario is a sequence of scenes that are linked by the actions of the actors and events that occur during the scenario [23].

Menzel et al. further classify scenarios by three levels of abstraction into functional, logical, and concrete scenarios. Functional scenarios are described by language. They define the actors that are part of the scenario, their actions, and relationships and rely on a shared and consistent vocabulary. Logical scenarios can be derived from functional scenarios and are defined by a set of parameters and their parameter distributions. These can include the speed and trajectory of a vehicle, the temperature, or road conditions. Interrelationships between parameters are also possible, for example, the speed of a vehicle can depend on the distance to the ego vehicle. Parameters can also be fixed and not have a parameter range. For example, map information such as the road curvature or the initial location for the ego vehicle may not require a parameter distribution. Concrete scenarios are derived from logical scenarios by assigning each parameter a value depending on its distribution. A single logical scenario can yield an unlimited number of concrete scenarios by choosing unique combinations of values for each realization [14].

2.1.1 Test Case Generation

Before scenarios can be tested, they need to be identified. There are two types of approaches for generating scenarios: knowledge-based and data-driven. Knowledge-based test case generation relies on expert knowledge to identify and specify challenging scenarios. Ontologies are commonly utilized to formalize scenarios as they provide a structured framework to represent knowledge about traffic participants, the environment, and the actions and relations between traffic participants [3]. However, methods in this category may be susceptible to human biases and may therefore fail to generate a representative set of scenarios [17].

Data-driven test case mining, on the other hand, relies on the clustering of recorded traffic data to identify logical scenarios. The data is predominantly in the form of recorded test drives, but may also be accident data, or other traffic-related data. Although termed data-driven, these approaches may still integrate expert knowledge. The test drives that are analyzed can either originate from real, on-the-road testing or from simulated drives. For real-world testing, vehicles are equipped with sensors and navigated by human drivers [16]. Scenarios that can be extracted from these datasets are only rarely interesting or challenging [11]. Therefore, the size of the collected data must be large enough to also include a sufficient amount of challenging scenarios. This, combined with the high cost of equipping the vehicle with the necessary sensors, makes this approach very expensive [5].

As an alternative, data can be collected using virtual test drives by simulating the behavior of the ego vehicle and all other traffic participants. Using this method, large amounts of data can be collected in little time. However, since traffic participants are only simulated, their behavior and therefore also the interactions between the actors may not be as complex or as representative as those that could be observed between real drivers. The data and the identified scenarios may therefore be of lower quality compared to data collected using real drives. Several intermediate steps are possible, where some actors are navigated by human drivers. This improves the quality of the data, but data collection is slower overall [5].

After the data has been collected, it needs to be analyzed in order to identify the scenarios that it contains. Cai et al. classify scenario identification techniques into three types: rule-based, relying on unsupervised machine learning, and relying on supervised machine learning. Rule-based methods use a set of predefined rules that are determined using expert knowledge. For example, a sequence or combination of several elementary activities or states, such as braking, acceleration, or deceleration, may constitute a new scenario. States may be identified by defining thresholds or parameter ranges for individual parameters. This approach is suitable

for simple scenarios with clear-cut parameters but may struggle with more complex scenarios and may also be limited by human bias. Unsupervised machine learning techniques identify patterns and scenarios without relying on previously labeled data and use algorithms such as K-Means, K-Medoids, and hierarchical clustering. These techniques can uncover unknown or complex scenarios by analyzing the inherent structure of the data. When using supervised machine learning techniques, models are trained on labeled data to recognize specific scenario patterns. This approach combines expert knowledge with machine learning to efficiently recognize scenarios but requires precisely labeled training data [5].

2.1.2 Scenario Based Testing

After logical scenarios have been identified, they can be tested. Given a logical scenario, concrete scenarios are generated based on it by assigning each parameter a concrete value. The strategy of how the parameter values are chosen may depend on the testing objective. If the objective is to evaluate the performance of the ADS on the given scenario, then the parameter space may be sampled at random. The concrete scenario can then be tested and evaluated individually. If the testing objective is to find a concrete scenario that meets certain criteria, such as the ego vehicle's collision with another vehicle, adaptive approaches may be used where each successive test result influences the generation of the next concrete scenario. After a logical scenario has been parameterized, it is tested and evaluated [23].

Testing is usually performed in a simulated environment that tries to emulate the real driving conditions as closely as possible. This approach is called software-in-the-loop testing. Popular open source simulators include Carla and AIRSIM, while large car manufacturers and other providers of commercial ADS technology usually rely on proprietary simulation software [23]. It is also possible to test scenarios on a track in the real world by trying to replicate the scenario specification as closely as possible. This approach is, however, less reproducible, much more expensive, and potentially not feasible in a safe manner [9]. Between software-in-the-loop testing and testing in the real world, several intermediate steps, such as hardware-in-the-loop testing, exist, where some physical components of the vehicle, such as sensors, are integrated with the simulated testing environment [23]. To determine how well the ADS performed, evaluation criteria are defined. A popular metric for the performance of a test is the time to collision (TTC), which measures the time that it takes, if any, for the ego vehicle to collide with another vehicle [22].

2.2 Trace Graphs

Trace Graphs are a method of data-driven test case mining that is both unsupervised and requires no expert knowledge [15]. It transforms the sensor data into a condensed graph where each node corresponds to an abstract situation that was encountered during a test drive, and each edge corresponds to a transition between two abstract situations. Paths in the graph constitute possible scenarios. Similar situations are identified by partitioning each individual sensor value range into a fixed set of bins and grouping those situations whose sensor values are assigned to the same bins. Minor differences between situations are therefore discarded. If similar situations were not grouped, each node in the graph would correspond to a single, unique situation, and it would be impossible to identify common scenarios. Also, differences that occur simply due to inaccuracies of the recording sensors are ignored this way. Because the approach is graph-based, it allows for the efficient comparison of new test data with an existing scenario catalog. Regular coverage criteria can be used to assess if the abstract situations that are contained in the new data are already known or if they are yet to be tested. Therefore, Trace Graphs serve two benefits. They allow for the identification of scenarios, i.e., test case mining, and also provide a way to calculate test coverage.

2.2.1 Traces

The starting point for the construction of a Trace Graph is the raw data collected using sensors during one or multiple test drives. It may include properties such as the speed of the vehicle, the rotation angle of the steering wheel, temperature, lighting conditions, etc. Each test drive yields a time series where each timestep corresponds to a concrete driving situation, which is the collection of all the measurements that were recorded at that specific point in time by the vehicle's sensors. This time series is referred to as a trace. It can be described as an $(m \times n)$ matrix of m timesteps with n sensor measurements each. To abstract from the physical nature of the sensors that collect the data and to adhere to the naming scheme proposed by Ulbrich et al. [21], each sensor is instead referring to a parameter, also called a trace parameter. Each parameter corresponds to a vector of m time-dependent parameter values.

2.2.2 Discretization

The first step in the transformation of the input-traces to a Trace Graph is the discretization of the driving situations. The value range of each parameter is partitioned into bins, which represent parameter value intervals by defining a set of bin

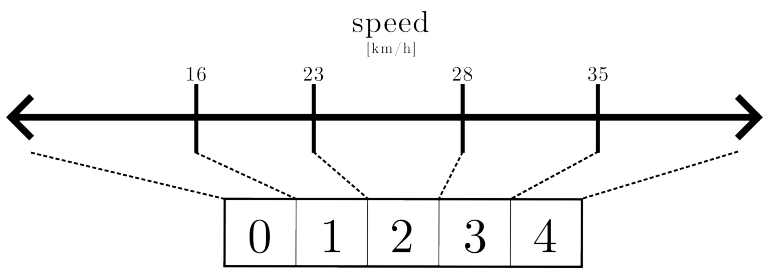

Figure 2.1: A mapping for the speed parameter consisting of five bins

borders. The first and last bins are open-ended, meaning that they cover all values that are smaller or larger than the first or last bin border, respectively. Thus, by defining n interval borders, $n + 1$ bins are created. The bins are identified by their index. The first bin is assigned index 0, the second index 1, and so on. Each parameter value is assigned to one bin, depending on which bin contains the value. This mapping can be seen as performed by a mapping function that takes as input a parameter value and outputs a bin. Figure 2.1 shows a mapping for a speed parameter that consists of five bins. Values in the range $[-\infty, 16[$ are mapped to bin 0, $[16, 23[$ to bin 1, and so on up to bin four, which corresponds to values in the range $[35, \infty]$.

There is one mapping function for each parameter, and the set of all mapping functions is referred to as a Parameter Discretization Model (PDM). The PDM can be applied to a situation by applying each mapping function to the corresponding sensor measurement, thereby yielding a state. The partitioning must neither be too coarse nor too fine-grained. If the partitioning is too coarse, too much information may be lost as situations that are inherently dissimilar will be grouped. On the other hand, if it is too fine-grained, similar situations will not be recognized as such. The method that Metzger et al. used in their paper is detailed in section 2.2.5.

The result of applying the PDM to each situation in a trace is an m-sized sequence of states, where a state is an n-sized vector of integer bin values. Each "concrete situation" in the time series is mapped to an "abstract state" in the state sequence. Each situation in the time series is mapped to one state. On the other hand, due to the condensation of the data by applying the PDM, a state may correspond to multiple situations, which can span multiple traces. This property will be used during graph construction in the next step. Because multiple successive situations may be mapped to the same state, a trace can be split into continuous ranges of situations, with each situation within a range mapping to the same state.

A subsequence of a trace will be referred to as a subtrace. Just like a regular trace, a subtrace can be transformed into a state sequence. As the next section will show, a state corresponds to a node in the graph. Therefore, the state sequence of a subtrace forms a path. While there exists a path for every subtrace, not every path in the final graph has a corresponding subtrace.

2.2.3 Graph Construction

The result of applying the PDM to each input trace is a set of state sequences. The final Trace Graph is constructed by creating one node for each unique state and one directed edge for each unique transition from one state to another. Because one edge is generated for each unique transition, the graph will not be a multi-graph, even if duplicate transitions exist. Because states in the sequence are not unique, as described in the previous section, it will likely have fewer nodes than the sum of the lengths of the state sequences. Self-loops are excluded, even if a state sequence contains a transition between identical states. The number of times that a state repeats is not lost, however, but will be kept as auxiliary information that is tied to each node. There is a one-to-n relation between situations and states and a one-to-one relation between states and nodes. Therefore, a situation also corresponds to a node. In the following, a node will be said to "contain" or "possess" states and situations, i.e., "a node's state" or "a node's situations" refers to the state or the situations that map to the node.

Figure 2.2 shows the construction of a Trace Graph based on a single input trace consisting of 8 situations. The trace is based on the readings of two sensors, one for the speed and the other for the steering wheel angle. The mapping function for the speed parameter is the same as in figure 2.1. The steering wheel angle parameter uses 5 bins based on the intervals $[-\infty, -9[$, $[-9, -3[$, $[-3, 3[$, $[3, 9[$, and $[9, \infty]$. The resulting Trace Graph has four nodes because there are four unique states in the state sequence.

Auxiliary information that can be stored for each specific node includes references to the situations that correspond to a node and the traces that contain said situations. Each node also has a frequency and a visit duration. The frequency describes how often the state is reached in the state sequences, while the visit duration describes how often the state is repeated after being reached. In figure 2.2, the node corresponding to state $(1, 3)$ has a frequency of 2 and a visit duration of 0 since the state does not repeat. The node for state $(3, 2)$ also has a frequency of 2, but a visit duration of 1, because the state repeats once in the state sequence. For edges, the source and target states, their traces and situations, and the number of times an edge is traversed can be stored. Frequency, visit duration, and traversals can all

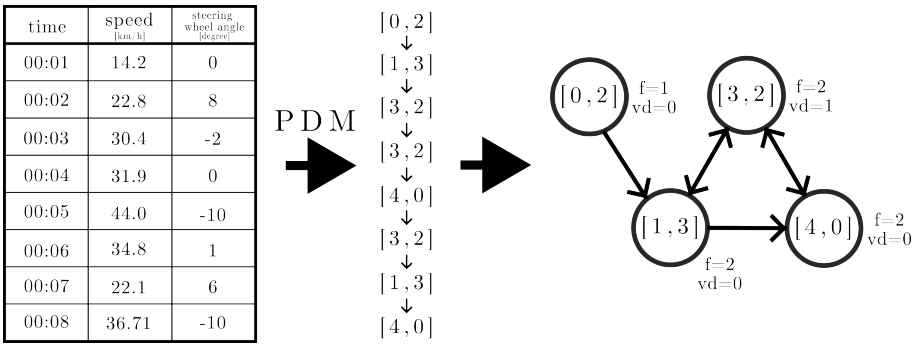

Figure 2.2: The construction of a Trace Graph from a single input trace

be computed from the situations of a node or edge, respectively. It would therefore be sufficient not to store them explicitly but to compute them when needed. The auxiliary information that is stored for nodes and edges can be used to assign importance scores, which may be used to distinguish important from unimportant nodes in a visual representation of the graph.

2.2.4 Graph Comparison

For the purpose of test case mining and computing test coverage, two Trace Graphs, a base graph and a delta graph, can be compared. The base graph represents the test scenario catalog, which contains the traces that are already known. The delta graph consists of traces that may contain unseen states and transitions. First, the base and the delta graphs are merged. The new graph contains every node and every edge of both graphs. Each node and edge in the base and delta graphs belongs to one of three disjoint subgraphs of the union graph. The base-only subgraph contains nodes and edges that only exist in the base graph. The base-delta subgraph consists of all nodes and edges that exist in both the base and the delta graph, and the delta-only subgraph contains nodes and edges that are only present in the delta graph. States and transitions that have not yet been seen are part of the delta-only subgraph. States that are in the base-delta or base-only graph are part of traces that are already known.

Common test coverage scores can be computed by comparing the delta graph to the base-delta subgraph. Node coverage can be computed by dividing the number of nodes in the base-delta subgraph by the number of nodes in the delta graph. This measure gives the share of states in the delta data that are already part of a known trace. Alternatively, edge coverage can be computed by dividing the number of edges in the base-delta subgraph by the number of edges in the delta graph. Other

test coverage criteria can be applied similarly.

Next, traces from the delta graph can be selected such that a given coverage criterion is satisfied. For example, in the case of node coverage, traces need to be chosen such that every node in the delta-only subgraph is contained in at least one of the selected trace's state sequences. The same can be applied for edges. Different heuristics, such as minimizing the combined length of the selected traces, can be used to choose a representative set. Finally, the selected traces can be added to the base graph for future comparisons.

To identify the scenarios contained in a Trace Graph, Brinkmann et al. introduce the concept of normal states. A normal state is a state that "[acts] as [a] natural [boundary] for scenarios". The ego vehicle tends to frequently encounter and "gravitate" toward normal states, a concept that they refer to as *attraction*. Once a normal state has been reached, it tends to repeat itself, which they refer to as *stability*. To identify normal states, they measure a state's *attraction* and *stability*, multiply both values, and then determine a cutoff value to separate normal from non-normal states [4].

Then, the state sequences are examined for possible scenarios. A subsequence of a state sequence qualifies as a scenario if it starts and ends at a normal state. Scenarios identified by this method may have a high degree of similarity. For example, two scenarios may differ only by a single state and be otherwise identical. To reduce the total number of scenarios while ensuring a certain level of diversity in the final set, they are clustered using Agglomerative Hierarchical Clustering (AHC), and then a random scenario is chosen as a representative from each group [4].

2.2.5 PDM Optimization

PDMs are generated in a two-step process. First, potential bin borders are identified, and then a genetic algorithm decides which bin borders will be adopted for the final PDM. To find potential bin borders, the histogram of each individual time series of sensor recordings is interpolated, and the resulting interpolation function is analyzed for local minima, local maxima, and local maxima of its first derivative. While the resulting PDM performs well at identifying differing situations, it is too fine-grained, and similar situations are not aggregated to a sufficient degree. Therefore, a genetic algorithm is used to reduce the number of bin borders.

A random bit string is created for each parameter with a length that equals the number of potential bin borders. A 1 signifies that the respective bin border is part of the final PDM, while a 0 means that it is removed. The individual bit strings are

then concatenated into one larger bit string representing all possible bins across all parameters. This procedure is repeated for a fixed population size. Reproduction is conducted by crossover, where each bit of a pair of bits is equally likely to be included in the offspring. The selection depends on a fitness function that assigns a score based on coverage analysis.

The calculation of the fitness function is based on a set of reference drives and a set of free drives. Reference drives are recorded on the same route and should therefore contain similar situations. Free drives are all recorded on different routes, so each free drive is expected to be different from both the reference drives and the other free drives. If the size of the set of reference drives is n, then n so-called delta values (DV) are calculated and averaged. For each calculation of a delta value, a different reference drive is selected, and a Trace Graph is calculated from it by transforming the bit string that is evaluated into a PDM. This graph will act as a delta graph and be compared to the base graph, which is constructed from all remaining reference drives. Additional delta graphs are constructed from each individual free drive. They will also be compared to the base graph. The value that is measured in each base-delta comparison is the number of nodes in the delta-only subgraph of the combined graph divided by the total number of nodes in the delta graph. It is referred to as the delta ratio (DR). The DR is high if the base and delta graph are dissimilar because then the delta graph contains a large number of nodes that are only present in the delta graph and not in the base graph.

The DV is calculated by averaging the delta ratios of the free drives and subtracting the delta ratio of the reference drive. A high delta value signifies that the states which are identified from the free drives are different from those of the reference drives and that the states that are identified within the reference drives are similar. This reflects the assumption that base drives are similar and free drives are unique and therefore provides a good metric for the ability of the PDM to aggregate similar situations and differentiate dissimilar ones.

2.3 OSGi

OSGi is a specification for the dynamic deployment and management of Java-based modules called bundles [18]. It manages how bundles are started and stopped, what services they provide to other bundles, which packages are accessible or hidden from other bundles, and more. The OSGi Alliance, which included members such as IBM, Motorola, and Sun Microsystems, developed the OSGi specification until November 2020, when it transferred ownership of the project to the Eclipse Foundation, specifically the OSGi Working Group, to maintain and evolve the project

[8]. Eclipse also develops the reference implementation Equinox and uses OSGi as the basis for plugins for its Eclipse IDE. It is also used by Cytoscape for the same purpose.

The project is divided into several layers, including 1) the module layer, which provides a modularization model for Java applications, something that Java itself did not have until the release of Java Modules with Java 9; 2) the life cycle layer, which handles how applications are installed, started, stopped, uninstalled, etc.; and 3) the service layer, which handles how bundles can provide services to other bundles.

2.3.1 Module Layer

An OSGi bundle is a JAR (Java archive) that contains Java class files and resources, with an additional manifest file that contains information about what packages the bundle requires to work properly (imported packages), what packages to expose to other bundles (exported packages), and additional meta-information such as the bundle name, entry point, version, and so on. Bundles are self-contained units that can be installed, started, stopped, updated, and uninstalled at runtime. Before a bundle is started, the OSGi runtime resolves any dependencies that the bundle declares in the "Import-Packages" header of its manifest file by matching them against the "Export-Package" declarations of other bundles. Using the information collected during dependency resolution, the importing bundle's class loader can then load any necessary dependencies. Each bundle having its own class loader ensures that bundles are isolated and avoids the risk of classpath conflicts. It also allows different bundles to use different versions of the same package at the same time.

With the release of Java 9, Java introduced its own module system for declaring package dependencies and exported packages. However, whereas OSGi provides dynamic modularity at runtime, Java's module system focuses on static modularity at compile time. It does not support functionality equivalent to the lifecycle and service layers.

2.3.2 Life Cycle Layer

The life cycle layer handles how bundles are installed, started, stopped, updated, and uninstalled. A bundle is initially in the INSTALLED state. If the OSGi runtime is able to successfully resolve all dependencies of the bundle as described in the previous section, it moves to the RESOLVED state. Once a bundle has been resolved, it can be started and activated. To do this, the bundle manifest contains

a *Bundle-Activator* header that must point to a class that implements the *Bundle-Activator* interface. This interface has a *start* method that takes a *BundleContext* object. The *BundleContext* allows the bundle to register services, retrieve references to services, and install other bundles at runtime. When the bundle is started, it enters the STARTING state. After the *start* method has been successfully executed, the bundle is set to the ACTIVE state.

When the bundle is stopped by calling the *BundleActivator*'s *stop* method, it must release any resources it acquired during its runtime. However, services that it has registered are automatically freed. If it is successfully stopped, it returns to the RESOLVED state. From there, it can be restarted by calling the *start* method or uninstalled, which will move it to the UNINSTALLED state.

2.3.3 Service Layer

A service is an object that a bundle can register under one or more interfaces with the Framework Service Registry, which is the central repository that handles the mapping between service interfaces and the implementing objects. Other bundles within the OSGi environment can then dynamically request these services at runtime by specifying the interface that the service implements. When registering a service, properties can be defined to provide additional information, such as the version of the service. Bundles can then request services based on these properties.

Services promote loose coupling between components. Instead of relying on a concrete implementation of a particular interface, components interact with services only in the form of their interfaces. They therefore play a crucial role in the development of scalable and maintainable applications.

2.4 Cytoscape

Cytoscape is an open source software for the visualization of molecular and genetic interaction networks, originally introduced by Shannon et al. in 2002 [19, 20]. It has been funded by the National Resource for Network Biology (NRNB) and is currently maintained by a consortium of researchers from UC San Diego, UC San Francisco, and the University of Toronto. Although it is aimed towards biological research, it can be used to analyze and visualize networks of other domains, such as social networks or, as in this case, Trace Graphs.

2.4.1 Architecture

Cytoscape is developed utilizing the OSGi framework as its foundation. The application itself is a set of bundles that interact using the OSGi service model. It uses Apache Karaf, an OSGi-based container with features such as hot deployment of bundles and a command line console for bundle management. While Karaf can internally support various OSGi runtimes, the packaged version within Cytoscape specifically uses Apache Felix [20].

Cytoscape provides an API bundle that contains and exports all API packages. The implementations of the interfaces are contained in separate implementation bundles that import the API packages from the API bundle. For each API package, there is an *-impl* package that provides implementations and registers services of the interfaces defined in the API packages. For example, the implementations for the interfaces defined in the *model-api* package are provided in the *model-impl* package and its subpackages, which are deployed in the *model-impl* bundle. The *model-impl* bundle will register services for many of the interfaces defined in *model-api*, mainly for factory and manager classes. Other bundles will then be able to request and use these services.

Because it relies on the OSGi framework, Cytoscape provides a rich environment for the development of plugins that allow developers to provide support for additional methods of analysis, layouts, or file formats, among other functionality. Plugins, which are referred to as "Apps" by Cytoscape, are also deployed as OSGi bundles and are similar to implementation bundles. They also have access to the API packages via the API bundle and can access the services provided by the implementation bundles. They can be dynamically loaded by either downloading them from the application's app store or by simply placing them in the correct directory, where they will be picked up and loaded by Karaf.

To separate the definition and implementation of classes, Cytoscape makes extensive use of the factory pattern. Many of the core interfaces of the Cytoscape model, such as tables, networks, network views, etc., do not have implementations that are accessible outside of the bundle that implements them. Rather, the packages that implement these interfaces also provide implementations for factory classes that contain methods to instantiate such an object. For example, the *model-api* package contains a *CyTable* interface, which is implemented by the *model-impl* package as *CyTableImpl*. The API also defines a *CyTableFactory* interface, which is implemented as *CyTableFactoryImpl*. *CyTableFactoryImpl* is registered as a service for the *CyTableFactory* interface on startup by the *model-impl* bundle. When a class wants to create a *CyTable*, it requests the *CyTableFactory* service and calls

its *createTable* method, which returns an instance of type *CyTable* (specifically, *CyTableImpl*). Therefore, no packages but those contained in *model-impl* will have access to *CyTableImpl* or be able to instantiate an instance of *CyTableImpl* on their own.

The state of the application is controlled by singleton *Manager* objects, which are also registered as services. They can be used to add new networks and tables, change how the data is mapped to its visual representation, or manage sessions. Some important managers in the application are:

- *CyApplicationManager*, for managing the overall state of the application, such as the currently selected network, network view, directory, etc.
- *CyNetworkManager*, for adding and destroying networks
- *CyNetworkViewManager*, for adding and removing network views
- *VisualMappingManager*, for managing visual styles and mapping visual styles to network views

2.4.2 Tasks

Cytoscape provides an environment to create and execute parameterized tasks that can be run asynchronously. The *Task* interface provides methods for starting and cancelling a task. The *start* method receives a *TaskMonitor* object, which can be used to set status messages or the task progress, which are displayed in a popup window to the user. If an exception is thrown in the task and the task does not catch it, the error message is also shown.

Tasks are typically not instantiated independently but rather through a *TaskFactory* that creates and configures them. By registering a *TaskFactory* as a service, a menu item is created in the GUI. When the user clicks the menu item, Cytoscape uses the *TaskFactory* to create a task and then runs it. Because task factories usually require the same input parameters to create and configure tasks, Cytoscape provides a range of predefined *TaskFactory* interfaces. For example, a *NetworkViewTaskFactory* can create tasks that operate on a network view, while a *NodeViewTaskFactory* receives a node view as its input. Depending on the type of task factory that is registered, the menu item may appear in different locations of the application. By registering a simple *TaskFactory*, the menu item will appear in the main menu bar. If a *NetworkTaskFactory* is registered, the menu item is displayed when the user right-clicks on the background of a network view. The network view that the user clicked on is then passed to the method for creating tasks. Similarly, if the user right-clicks on a node, it will provide the node view to the *NodeTaskFactory*. Task

factories also provide a *isReady* method that will signal to the application whether it is ready to produce a task or not. If it is not ready, the menu item is grayed out and the user cannot click it.

2.4.3 Events

Cytoscape has an event system based on the OSGi service model that can notify interested objects about events in the model or view. If a class wishes to receive notifications regarding a specific event, it can do so by implementing the corresponding listener interface and registering itself as an OSGi service of the same interface. The class that creates the event will listen for registrations and deregistrations of services of that type and send the event to all listeners by calling their *handleXXX* method with the payload. For example, if a class wants to be notified about a change of the currently selected network, it can implement *SetCurrentNetworkListener* and register as a service for that interface. *CyApplicationManager* will be notified about the registration and, in turn, notify the object the next time the currently selected network changes.

2.4.4 Data Model

Cytoscape has two primary types for storing data: networks and tables. Networks, with the primary interface being *CyNetwork*, contain nodes and edges. The edges can be directed or undirected, and there may be multiple edges between two nodes. Tables, the primary interface being *CyTable*, allow for storing tabular data. While networks and tables can be used separately, they are usually linked on network creation, meaning that a node or edge in the network corresponds to a row in one or multiple tables. Besides making it possible to associate complex data with the nodes in the network, the style of a node, for example, its size or color, may also be mapped to the values of a column. This mapping is usually performed using the nodes and edges SUID (Session-unique identifier), which all nodes, edges, networks, and tables possess. The SUID is generated when the object is created and will stay constant over the lifetime of the object.

The *CyTable* implementation used by Cytoscape can store a limited number of allowed types, including *Integer*, *Long*, *Double*, *String*, and *Boolean*. It can also store lists of the above types. In addition to a name that is presented to the user, each table column also has a namespace that is used to avoid naming conflicts with other applications. The column name, primary key type, and value type are set when a table is created. It is not possible to store objects of different types or custom types using the default implementation of *CyTable*. The default *CyTable* implementation has certain properties that make it unfavorable to store trace data.

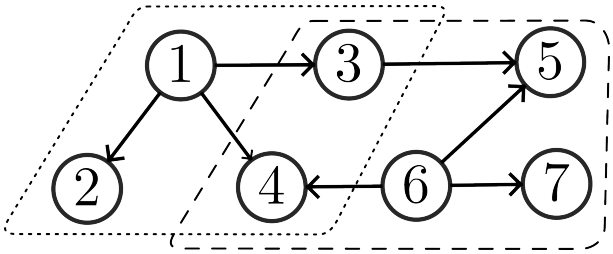

Figure 2.3: A root network that contains two subnetworks

For this purpose, a custom implementation is used, which is explained in section 3.6.2.

Networks are organized in a two-level hierarchy. There are root networks (*CyRootNetwork*) and subnetworks (*CySubNetwork*), both of which implement the *CyNetwork* interface. Each root network contains one or more subnetworks. When a node is added to a subnetwork, and it does not exist in the root network, it is also added to the root network. The same applies to edges. Because a root network can contain more than one subnetwork, it is common for each subnetwork to contain only a subset of the nodes and edges of its root network. When a node or edge is removed from a subnetwork and the subnetwork is the only subnetwork that contains that node or edge, it will be removed from the root network as well. If there are other subnetworks that contain a reference to the deleted element, it will not be removed from the root network. Figure 2.3 shows a root network containing 7 nodes that is split into two subnetworks. Nodes 1 through 4 belong to one subnetwork, and nodes 3 through 7 belong to the other subnetwork. Nodes 3 and 4 are contained in both subnetworks. All edges belong to either of the two subnetworks. It is not possible that an edge exists in the root network alone.

When a new network is created using the *CyNetworkFactory*, a new root network and a subnetwork are generated. The subnetwork is returned to the user, while the root network is only accessible by casting the *CyNetwork* to a *CySubNetwork* and calling the *getRootNetwork* method. This is only necessary if additional subnetworks need to be created or the root network's tables need to be accessed directly, which is rarely the case.

For every network that is created, several tables are also created and associated with it. The association between a network and a table is based on two parameters: a namespace and a graph component. The namespaces that are declared by default are "shared", "local", "hidden", and "default". Graph components include "node", "edge", and "network". The "shared" namespace is only used for tables that are

associated with root networks. One shared table will be created for every of the three aforementioned graph components. Each node and edge in the root network has a row in the shared node or shared edge table, respectively. The shared network table contains one entry for each subnetwork of the root network and can be used to store meta-information about the networks as a whole. The mapping between nodes, edges, networks, and the table rows is facilitated using the SUID. Shared tables are accessible by each subnetwork and therefore allow storing data that should be shared with other subnetworks.

The "local", "hidden", and "default" namespaces are used for tables that are associated with subnetworks. For the local and hidden namespaces, three tables are generated, one for each graph component. The difference between the two is that local tables are visible to the user in the application's GUI, whereas hidden tables are not. However, functionally, the two types of table are identical.

In order to streamline access to shared tables within the context of a subnetwork and facilitate data sharing between networks, default tables are available alongside local and hidden tables. These are facades that give access to the data of both the local table and the shared table at the same time. When the default node table for a subnetwork is accessed, the table has one column for each column in the local node table but also contains the columns that are defined in the shared node table. When a new column is created in the default table, it is actually created in the shared table, and a virtual column is created in the local table. Virtual columns map the content of a column in one table to another table by joining them based on a common value, similar to SQL join operations. Therefore, by creating a column in the default table, the data in these columns is accessible in other subnetworks as well. If the data should not be shared, the column must be created explicitly in the local table instead.

Figure 2.4 shows the same root network from Figure 2.3, but also displays the associated tables. For simplicity, only the node tables are shown. The shared table of the root network has seven entries, one for each node in the root network. The local tables of the subnetwork contain a row for each node in the respective subnetwork. The default node table of each subnetwork contains columns from both their respective local and the shared table.

The mapping between networks and tables is performed by the *CyNetworkTableManager*, which provides methods to associate and disassociate networks with tables. As described in the previous section, by default, there is one association per graph component and namespace pair. However, it is possible to define custom namespaces, which can be used to map additional tables. For example, a table can be associated with the node type that holds additional information about the nodes in

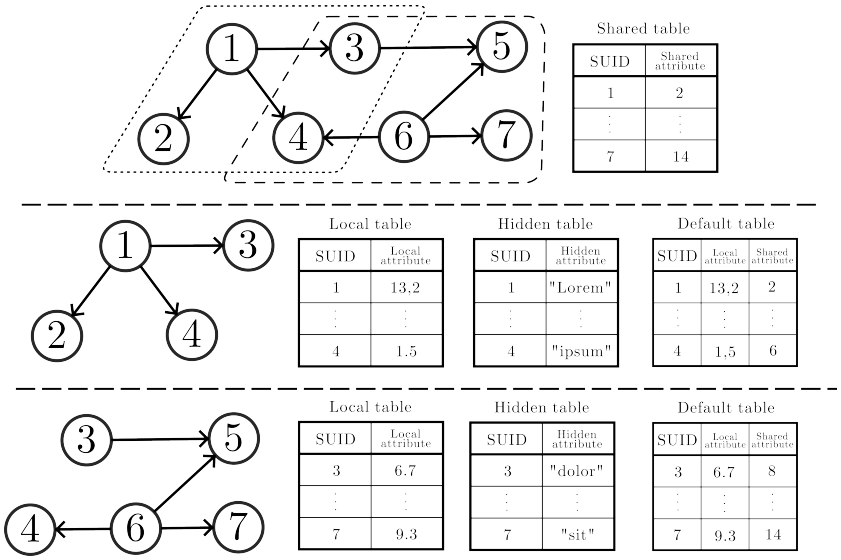

Figure 2.4: A root network, its two subnetworks, and associated node tables

the network. In this case, a node in the network is not necessarily linked to a row in the table, but the table data can still be presented to the user to provide further information.

2.4.5 Visual Styles

Each node or edge that is part of a network has a corresponding view object that represents its visual state. This is also true for the network object itself. The visual presentation of view objects in the GUI is determined by a set of visual properties. These properties include, for example, the size and color of a node, the width of an edge, or the background color of a network. Each property has a default value that will be used if a view object does not have a value set for a certain visual property using one of the three methods presented in the next section.

In order to customize the appearance of a network, visual styles can be created and assigned to the network. Whenever a change is made to a network, the visual style is automatically applied. Styles can contain default values and mappings, which are functions that calculate the value of a visual property based on the values of a table column. The next section will provide an explanation of the different mapping types. In case a visual property of a view object should not be affected by the visual style, a locked value can be set as a bypass for that specific object. This lock can later be lifted, and the visual property of the view object will again be determined

based on the visual style. To summarize, there are four ways in which a visual property for a view object can be set: the locked value has the highest priority, followed by the mappings and default values of a visual style. The default value of a visual property has the lowest priority.

There are three types of mappings: "discrete", "continuous", and "passthrough". A passthrough mapping forwards the column value to the property without modifying it. Therefore, the table column must contain valid and sensible values for the visual property. If the table column does not contain a value for a view, the default value of the visual style is used. A discrete mapping allows every input value from the table column to be mapped to a unique property value. For example, if a table column contains n different values, then each value can be mapped to a different node color. The continuous mapping is defined by a set of points that are piecewise linearly interpolated and can therefore only be used if the input type is numeric. Logarithmic functions and other arbitrary functions can be approximated using this type of mapping. This is useful for properties such as the node size. While the input type must be continuous, the output type may also be discrete. For example, it is possible to map all input values up to a certain threshold to one node shape and all values above the threshold to a different node shape.

Chapter 3

Contribution

This chapter presents the requirements for the application and the algorithms, design choices, and trade-offs that were made to implement them. The primary goal was the design of an application that would provide a clean and scalable visualization of Trace Graphs, as well as the tools necessary to analyze them in a variety of ways. The next section describes the requirements in detail.

3.1 Requirements

The first set of requirements is based on the exploration of Trace Graphs. The Trace Graph should be presented in a way that makes it easy for the user to interact with and understand the basic structure of the graph. This includes the ability to pan and zoom in and out of specific regions of the graph. Important nodes must be made distinguishable from unimportant nodes by setting their size and color based on characteristics such as the frequency and visit duration. Edge importance can be measured by the number of times that the corresponding transition appears in the state sequences and can define the width or color of an edge. The user must be able to select certain parameter ranges so that only nodes with parameter values within the user-defined limits are visible. Nodes with parameter values outside the ranges should be hidden. The nodes must also be automatically arranged so that they do not overlap too much.

The input data is provided in the form of traces. The application must therefore perform the discretization and graph construction, rather than receiving and simply rendering the already constructed Trace Graph. It should also be possible to provide the application with a PDM that defines the bins of all parameters. This will

require a data format to store the PDMs. The bin borders of the PDM should also be dynamically adjustable at runtime, and when a bin border is changed, the tool should update the Trace Graph and its visualization accordingly.

For the purpose of analyzing the Trace Graph and gaining a deeper understanding of the relation between traces, state sequences, and the graph, when a node is selected, the tool should highlight the subtraces of a certain size that have a state of the selected node at their center. When multiple nodes are selected, the tool should be able to find and display the path of the shortest subtrace connecting all the selected nodes. Also, when a node or edge is selected, information should be displayed that provides insights, such as the number of times a node or edge appears in the state sequence. Finally, two Trace Graphs should be comparable, and nodes in the base-only, delta-only, and base-delta subgraphs should be visually distinguishable.

3.2 General Architecture and Design

This section provides a brief overview of the general architecture of the application, its main classes, and how they interact. It does not provide details about the features but rather focuses on the architecture and design choices. The features and their implementation are explained in detail in the subsequent sections.

3.2.1 Points of User Interaction

There are three different ways in which the application can receive user input. The first is when the user interacts with a network view by selecting a node or edge. Controllers can receive notifications of these events by implementing listener interfaces and registering as services for those interfaces. To receive node and edge selection events, the class must implement and register as a *SelectedNodesAndEdgesListener*. The interface defines a *handleEvent(SelectedNodesAndEdgesEvent)* method that is called when one or more nodes or edges are selected or deselected. The second point of interaction is when the user clicks on a menu item. When the menu item is clicked, Cytoscape creates a *TaskIterator* using the associated *TaskFactory* and executes it. When the task is executed, the controller for the Trace Graph on which the action was performed is identified and its handler method is called. The third method is when the user interacts with the application's custom Swing panels. These panels have a reference to a controller and will call the appropriate handler method when an action is performed.

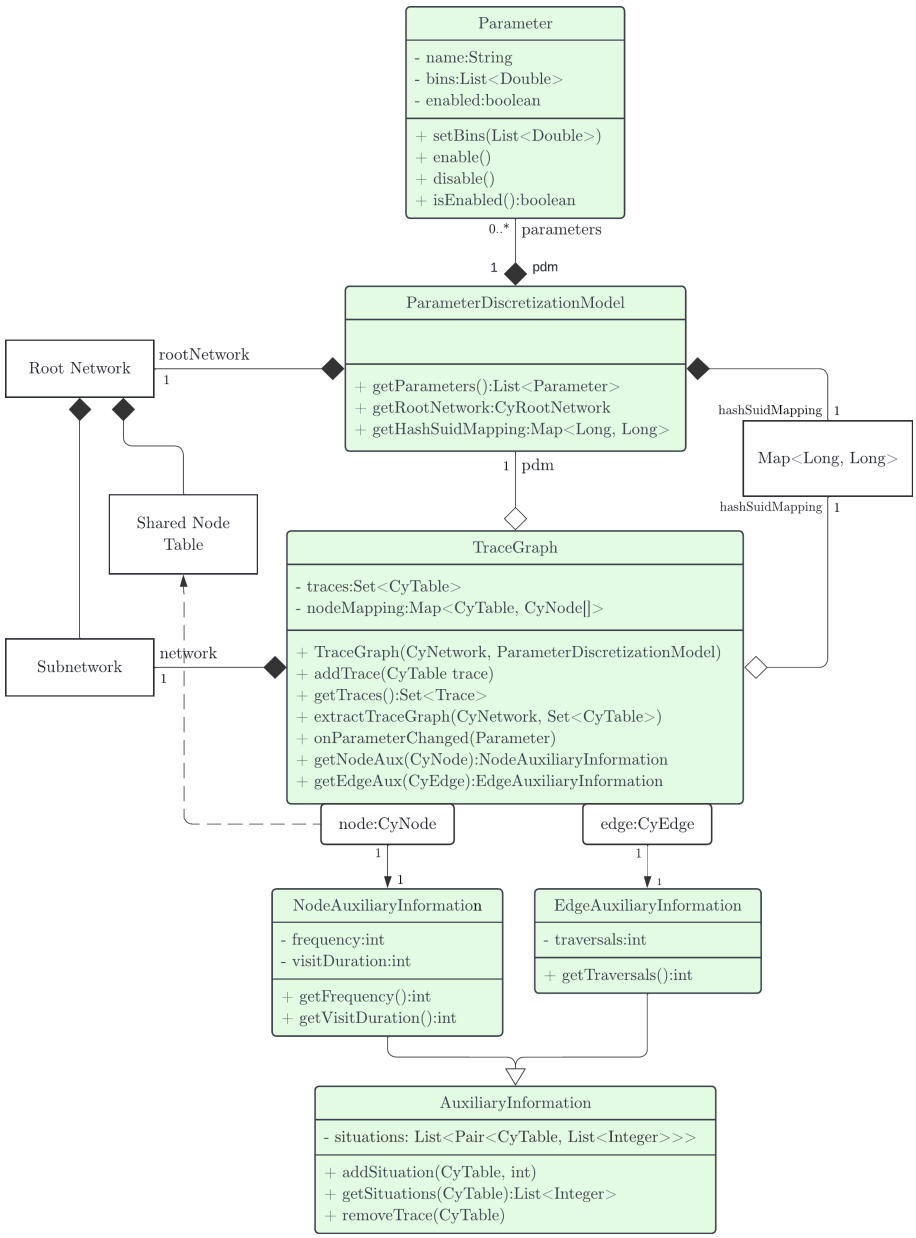

Figure 3.1: UML class diagram of model classes

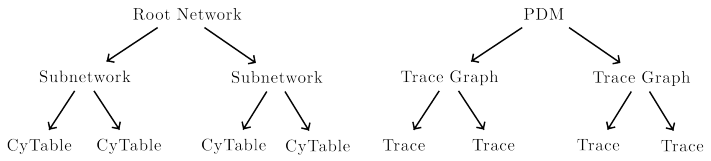

Figure 3.2: Cytoscape network hierarchy and corresponding Trace Graph objects

3.2.2 Model

Figure 3.1 shows a simplified UML diagram containing the most important model classes. Fields and methods that are not necessary to understand how data is stored and accessed are excluded. Classes that are specific to Cytoscape are further simplified.

In the application, the relationship between Parameter Discretization Models and Trace Graphs is not one-to-one but one-to-n. Each Parameter Discretization Model corresponds to a root network, and each subnetwork of the root network corresponds to a Trace Graph with at least one trace. A node can be part of several Trace Graphs. This has two main advantages. First, a node's state, which is the same regardless of which Trace Graphs it belongs to, can be shared by storing it in the root network's shared node table. This means that state information does not need to be duplicated across all Trace Graphs but can be stored centrally. Second, because the Cytoscape user interface groups subnetworks that share the same root network, i.e., PDMs, it is easy for the user to see which Trace Graphs belong together and can be merged or compared. Figure 3.2 shows a possible hierarchy with a root network containing two subnetworks, each containing two *CyTables*. This corresponds to a PDM with two Trace Graphs and two traces each.

The *ParameterDiscretizationModel* class holds a reference to its root network, which contains all nodes and edges that exist in at least one of its Trace Graphs. The PDM also has a list of parameters. Each parameter stores a list of bin borders, its name, and a flag indicating whether the parameter is enabled or not, as well as a reference to the PDM itself. It also provides a method for setting new bin borders. In response, objects that subscribe to the parameter are notified of the change and can react to it.

When a *ParameterDiscretizationModel* is created, the shared node table of its root network is populated with a column for each parameter. Then, when a trace is added to one of its Trace Graphs and new nodes are created in response, the state information for the new nodes can be stored in these columns. However, node and edge information that is specific to each Trace Graph must still be stored individu-

ally, either by storing it in the local node or edge tables, or by using auxiliary data structures that are not provided by Cytoscape.

A *TraceGraph* has a reference to its PDM, which it can use to query bins of parameters when constructing or updating the graph, the *CyNetwork* that contains its nodes and edges, and the *CyTables* that represent its traces. It also has maps from *CyNode* and *CyEdge* to classes that hold additional information about nodes and edges, called *NodeAuxiliaryInformation* and *EdgeAuxiliaryInformation*, respectively. To facilitate a mapping between situations and nodes, the Trace Graph maintains an array of *CyNodes* for each of its traces. The node corresponding to the situation at index i of a trace can be found at index i of the trace's array. Since each situation must always map to a node, the arrays are always kept up to date and will not contain empty or stale entries.

Traces are stored in a class called *Trace*, which implements the *CyTable* interface. When a trace is imported, its data is loaded into a new *Trace* instance, which can be added to a Trace Graph using the *addTrace* method. The *TraceGraph* stores the table, applies the PDM to the trace, and the graph is then updated accordingly. This process is explained in detail in section 3.3.1. The Trace Graph also has a method that can be called when a bin border of a parameter is added, moved, or deleted. When this happens, a situation may map to a different state, which means that the graph needs to be updated. A naive approach would be to delete all nodes, edges, and auxiliary information and reconstruct the graph from scratch. A more efficient approach to restoring the integrity of the graph is discussed in section 3.5.2.

In addition to loading a trace, a Trace Graph can also be split or merged with another Trace Graph. When two Trace Graphs are merged, the traces from a source Trace Graph are transferred to a destination Trace Graph. The source Trace Graph is subsequently destroyed and the transferred traces are initialized with the destination Trace Graph. When a Trace Graph is split, the opposite happens. A new Trace Graph is created, which receives and initializes a subset of traces from an existing Trace Graph. Splitting and merging Trace Graphs is explored in detail in section 3.3.3.

The auxiliary information that is stored for each node includes the node's visit duration and frequency as well as the situations that map to the node. Auxiliary information for an edge includes the number of traversals and the situations that map to the source state of the corresponding transition. The auxiliary information is kept up-to-date whenever the Trace Graph changes. Therefore, the visual representation of nodes and edges, which is tied to the auxiliary information, always reflects the latest state of the Trace Graph.

As a better alternative to the external *AuxiliaryInformation* classes, it would theoretically be possible to store the values in the local or hidden node or edge tables since data stored in local and hidden tables is not shared with other networks. However, due to performance issues caused by the Cytoscape search index, this was not a viable option. The search index is used internally by Cytoscape to facilitate its search functionality. Whenever a table row is added, modified, or removed, the search index is updated. Loading and updating a Trace Graph requires frequent updates to the auxiliary data. Therefore, if the data were stored in *CyTables*, the search index would cause a significant overhead for these tasks. The effects of the search index cannot be completely avoided, however, since creating and deleting nodes and edges and populating the shared node table with state information still require table edits. However, their impact is limited, as shown in chapter 4.

The *hashSuidMapping* that the PDM shares with all of its Trace Graphs is used to map from a state hash to a *CyNode* SUID. When a Trace Graph loads a trace or is updated due to a bin update, it can use the map to check whether a state already has a corresponding node in one of the other Trace Graphs that share the same PDM. If there is a match, it can use the SUID to retrieve the node from the root network and add it to its own subnetwork. If there is a miss, it will create a new node in its subnetwork, which will automatically create a node in the root network as well, populate its row in the shared node table with the state information, and insert the node into the *hashSuidMapping* using the state's hash as the key. If Cytoscape allowed the use of self-defined SUIDs, this process would be redundant as the hash could be used directly as the SUID. Since SUIDs are determined based on an incrementing counter, the mapping is required.

Figure 3.3 shows the relationships between the aforementioned model components. Situations 17 and 21 map to the node with SUID 237, while situation 24 maps to the node with SUID 314. Given a situation, the state can be determined by looking up the corresponding node in the node mapping of the Trace Graph, getting its SUID, and accessing the row in the shared node table that has the SUID as its primary key. It would also be possible to simply calculate the state each time, thereby making the node mapping redundant. However, because accessing a situation's state is a common operation, the node mapping allows for a significantly faster lookup in return for a small memory overhead. Given a state and assuming that there exists a node for the state, it is accessible by hashing the state and using the SUID from the hash-to-suid mapping to retrieve the node from the network.

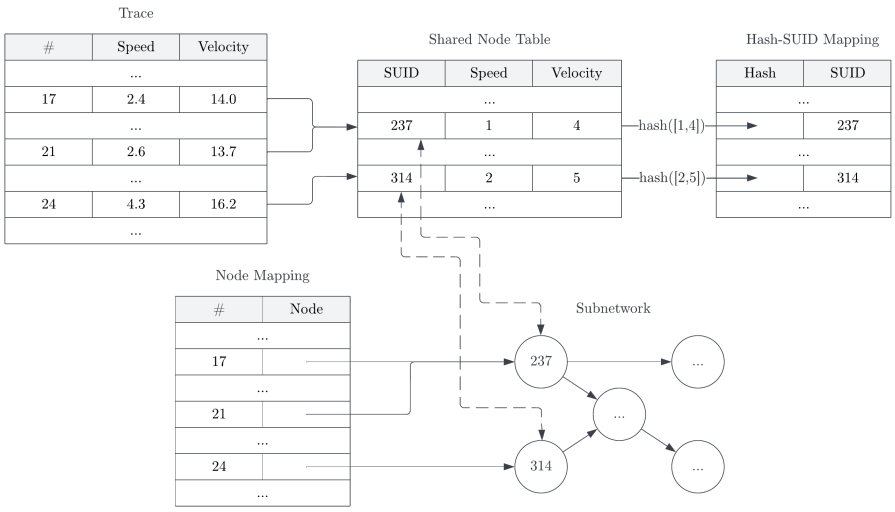

Figure 3.3: Model related components

3.2.3 Controller

Figure 3.4 shows a simplified UML class diagram of the application's main controller classes. Fields and methods that are not important to understand how the controllers are organized or interact have been omitted.

TraceGraphManager is a singleton object responsible for mapping between PDMs and *NetworkControllers*. It is registered as an OSGi service and is therefore accessible by getting a service reference via the bundle context. Classes that request the *TraceGraphManager* include instances of *NetworkTaskFactory*, which only receive a *CyNetwork* as input for constructing their tasks and require a way to resolve the *NetworkController*. For this, the *TraceGraphManager* provides a *findControllerForNetwork* method that iterates over all controllers in the PDM-to-NetworkController map and returns the corresponding *NetworkController* by comparing their associated networks to the network provided to the task. It also listens for *NetworkAboutToBeDestroyedEvents* that are fired when the user deletes a network. If the network is managed by the application, it will perform the necessary cleanup, which includes destroying the *NetworkController* and possibly removing the PDM if the destroyed network is the PDM's last remaining network.

A *NetworkController* is an abstract class that manages a network. It has two implementations. *TraceGraphController* manages a network representing a Trace Graph, while *NetworkComparisonController* manages a network representing the

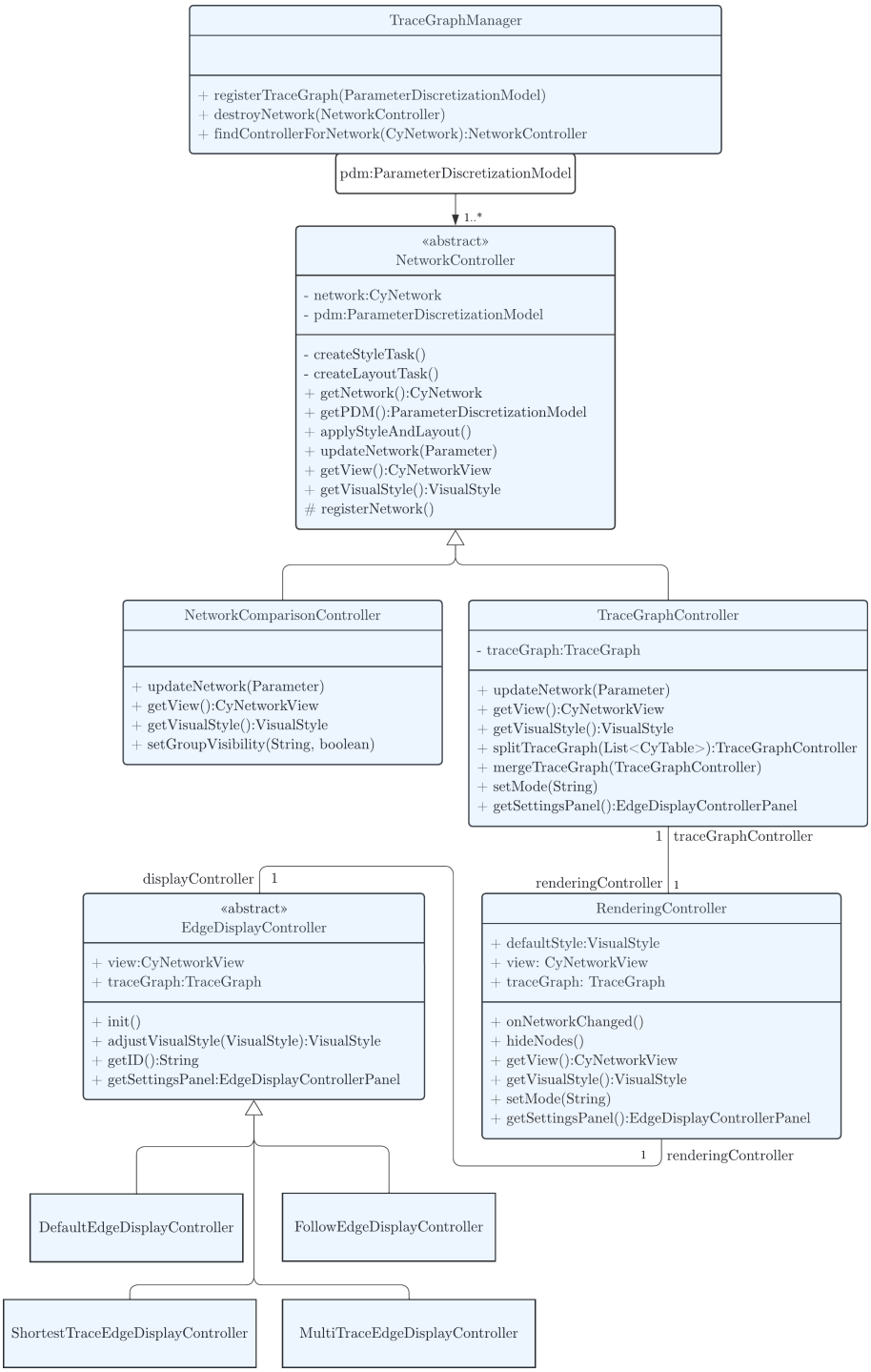

Figure 3.4: UML class diagram of controller classes

comparison of two Trace Graphs. A network controller is responsible for the styling and layout of its networks. It defines methods for creating style and layout tasks. It also holds a reference to the *CyNetwork* on which it operates but not to the *CyNetworkView*, which must be provided by the implementation. There are also a number of abstract methods that provide information about the graph, such as the name, number of nodes, number of edges, and so on.

A *TraceGraphController*, which is the more complex of the two *NetworkController* implementations, controls a Trace Graph. Due to its complexity, functionality related to node and edge styling and interactions with the network view has been delegated to a separate class called *RenderingController*. The *TraceGraphController* is only responsible for managing the state of the *TraceGraph* and ensuring its consistency. It offers methods that are called when an operation needs to be performed for the Trace Graph. In return, it will make the necessary adjustments and make a call to the RenderingController to update the view accordingly. The operations that it supports are updates to parameters, updates to the PDM, merges, or splits. Parameter updates include enabling or disabling parameters or an edit to a parameter's bins, which can be the insertion, deletion, or moving of a bin border. When a *TraceGraph* is merged or split, it ensures that the traces are correctly moved from one *TraceGraph* to another and that a new *TraceGraph* is created or an old one is removed as necessary. To be notified about parameter updates or updates to the PDM itself, it is registered as a listener for those objects. If a bin border of a parameter is created, moved, or destroyed, or if a parameter is turned on or off, it instructs the Trace Graph to update based on the changed parameter and then notifies the *RenderingController* of the changes, which updates the view.

The *RenderingController* is responsible for styling the network. It provides a method that the *TraceGraphController* can call when changes are made to the *TraceGraph*, upon which it will update the visual style and reapply the style and layout. It has an *EdgeDisplayController* that is responsible for styling edges and handling edge selection events. An *EdgeDisplayController* is synonymous with a mode that the *RenderingController* can be in. The *EdgeDisplayController* that is currently active, i.e., the rendering mode, determines which edges are visible or hidden, how the default style for the mode should be customized, what information is displayed in the mode's associated panel, and so on.

The second *NetworkController* implementation, called *NetworkComparisonController*, is concerned with managing the comparison of two *TraceGraphs* that share the same PDM. It renders a graph containing a union of the nodes and edges of both graphs. Since the nodes are stored in the root network, it can simply add the nodes of both graphs and no duplicate nodes are inserted. When the base or

delta networks change, it will not automatically update to reflect the changes in the comparison graph. Instead, the user must manually create a separate comparison. The comparison controller provides methods for hiding and unhiding base-only, delta-only, or base-delta nodes and edges.

3.2.4 View

Cytoscape allows Apps to display Swing-based panels by registering a *CytoPanelComponent* as an OSGi service. The tool registers such a panel but uses a tabbed panel that can display several different subpanels depending on the context. For example, when a node is selected, the panel creates and displays a subpanel containing information about the node. When the node is deselected and no other node is selected, it hides and removes the subpanel. It also listens for events that are triggered when a network that is controlled by a *TraceGraphController* or a *NetworkComparisonController* is selected. It then notifies its subpanels of the change, which unregister from the model of the previous controller, register as listeners to the new model, and update their components. This ensures that the panels always display information about the currently selected network.

The styling of the graph is handled by the RenderingController. It uses the *VisualStyleFactory* to create a *VisualStyle* and then adds mappings for the size of a node, the color of a node, and for a tooltip that is displayed when the user hovers over a node, which displays the node's state. Afterwards, it is registered with the network using the *NetworkVisualStyleManager*. When a different rendering mode is selected, the default style is passed to the rendering mode controller, which can make adjustments to it, such as setting new defaults or adding additional mapping functions. Cytoscape ensures that the visual style is always reflected in the visual representation of the graph by triggering re-renders periodically.

3.3 Exploration & Manipulation

3.3.1 Trace Graph Import

The application allows the import of both PDMs and traces. Traces must be provided in the form of CSV (comma-separated value) files with a header containing the names of the parameters. When a trace is loaded, the header allows the application to check if there is a PDM with the same set of parameters. The values must be formatted as floating-point numbers, optionally in scientific notation, and the first column must contain the row index.

A PDM can be provided to the application as a JSON (JavaScript Object Notation)

file containing fields for a name, description, parameters, and an optional list of traces to load initially when the PDM is loaded. Figure 3.5 presents an example of a PDM. The parameters are objects with a field for the name and a *bins* field. *bins* is a list of ascending floating-point numbers describing the bin borders. For example, the bin borders 0.5, 1.5, and 2.5 would result in 4 bins: $[-\infty, 0.5[$, $[0.5, 1.5[$, $[1.5, 2.5[$, and $[2.5, \infty]$. The optional *traces* field contains a list of paths to traces that will be loaded and added to a new Trace Graph when the PDM is imported.

Figure 3.6 shows the process of importing a PDM. The general procedure is determined by two tests. The first test checks if the PDM file references any trace files. The second test checks if there are any existing PDMs that have the same set of parameters as those listed in the imported file.

If both tests are true, the user is presented with a dialog that allows them to either select one of the matching PDMs or create a new PDM instead. If the selection is confirmed and an existing PDM is selected, the new parameter settings are applied to all of its existing Trace Graphs. Otherwise, if the user chooses to create a new PDM, it is instantiated using the parameter settings from the input file. The details of how to instantiate a PDM are deferred to the end of this section. A new Trace Graph is created using the selected or newly created PDM, and a new trace is created from each of the referenced trace files, which are then added to the Trace Graph. The *CyTable* used to store traces is not the default *CyTable* that is provided by Cytoscape. Instead, an optimized version, which is explained in section 3.6.2, is used. The algorithm for loading traces to a Trace Graph is explained in section 3.5.1.

If the parameters match those of an existing PDM, but the PDM file does not reference trace files, the option to create a new PDM is disabled because a root network must always contain at least one subnetwork. The user can still select one of the matching PDMs, and the parameter settings will be applied to the existing Trace Graphs. If there are no matching PDMs, but the PDM file references trace files, a new PDM is automatically created. The user is not presented with a dialog because there are no options to choose from. A new Trace Graph is then created and populated with the traces as described above. If the PDM file does not reference any trace files and there are no matching PDMs, an error is thrown. There must be at least one trace for each Trace Graph because a root network requires at least one subnetwork, and a root network is created for each new PDM.

Instead of importing a PDM file, it is possible to import a trace file directly. The process is similar to importing a PDM. The application checks to see if one or multiple Parameter Discretization Models with the same set of parameters already exist. If so, the user is presented with a dialog to select which PDM to assign the

```
{
  "name": "normal_speed_combined_cleaned",
  "traces": [
    "normal_speed_combined_cleaned.csv"
  ],
  "description": "signal data of a 4x4 position matrix
      plus speed and steering angle",
  "parameters": [
    {
      "name": "r_-3",
      "bins": [
        0.5,
        1.5,
        2.5
      ]
    },

    ...

    {
      "name": "steering_angle",
      "type": "double",
      "bins": [
        -0.00938785,
        -0.00258538,
        -0.00031098,
        0.00079522,
        0.00402804,
        0.01396263
      ]
    }
  ]
}
```

Figure 3.5: A JSON file describing a PDM

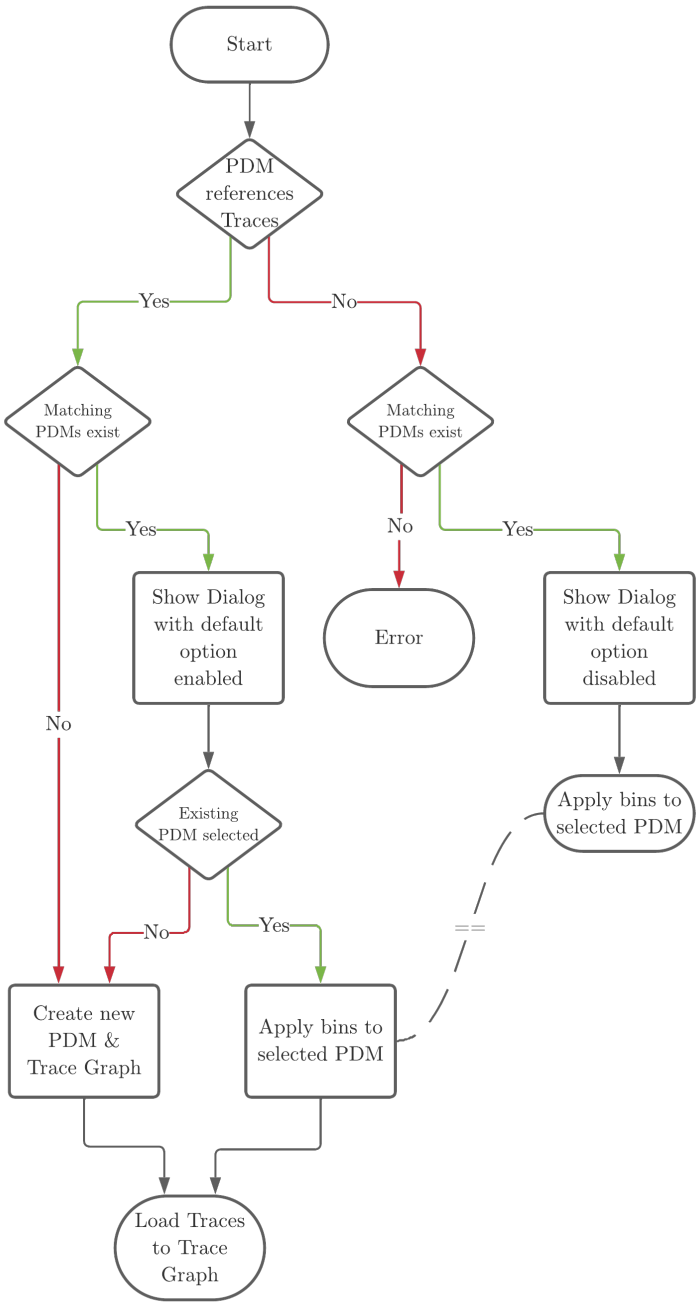

Figure 3.6: Flow diagram of importing a PDM

trace to. A new Trace Graph is then created for the selected PDM and the trace is added to the Trace Graph. Alternatively, the user can choose to create a new PDM without bin borders. This is performed automatically if no matching PDMs are found. If the user chooses to create a new PDM, the associated Trace Graph will have a single node and no edges. Each state calculated from the situations in the trace is mapped to the zero vector, so each situation is mapped to the same node. The user can then create bin borders manually or import a PDM to set the bins and the graph will update automatically. The PDM can also be exported at a later time.

Initializing a *ParameterDiscretizationModel* from a PDM file involves several steps. First, a *CySubnetwork* and its root network are created via the *CyNetworkFactory*. Then, the name of the root network is set to the name specified in the PDM file. The root network and the parameters from the PDM file are passed to the constructor of the PDM, and the subnetwork is used to initialize the associated Trace Graph. To mark the subnetwork as being managed by the application, a marker column is created in the local network table. Later, when other parts of the application want to test whether a network is managed by the application, they can check for the existence of the marker column. Given an already existing PDM, a Trace Graph can be created by passing it a subnetwork created from the PDM's root network.

3.3.2 General Workflow and Layout

Once a Trace Graph has been imported, the user can begin interacting with it. Figure 3.7 shows the general layout of the application. On the left, the default panel displays information about the currently selected Trace Graph, such as the number of nodes or the names and sizes of the traces belonging to the currently selected Trace Graph. It is always displayed when a network managed by the application is selected. The user can also view the parameters of the PDM, enable or disable parameters, or open the dialog for editing a parameter's bin borders. At the bottom, below the graph view, the node states are displayed in a table. Alternatively, since each *CyTable* containing a trace is associated with the subnetwork of a Trace Graph, the user can choose to view the raw trace data.

When the user selects a node, another panel is added that displays information about the node, such as its frequency, visit duration, and situations. When the node is deselected or another node is selected, the panel disappears. Right-clicking an empty area of the network view displays a context menu that allows the user to change the rendering mode. Depending on the mode, the user may be presented with an additional panel that provides information and allows configuration of the mode. These, along with the panel for node comparisons and Trace Graph comparisons, are explained in their respective sections.

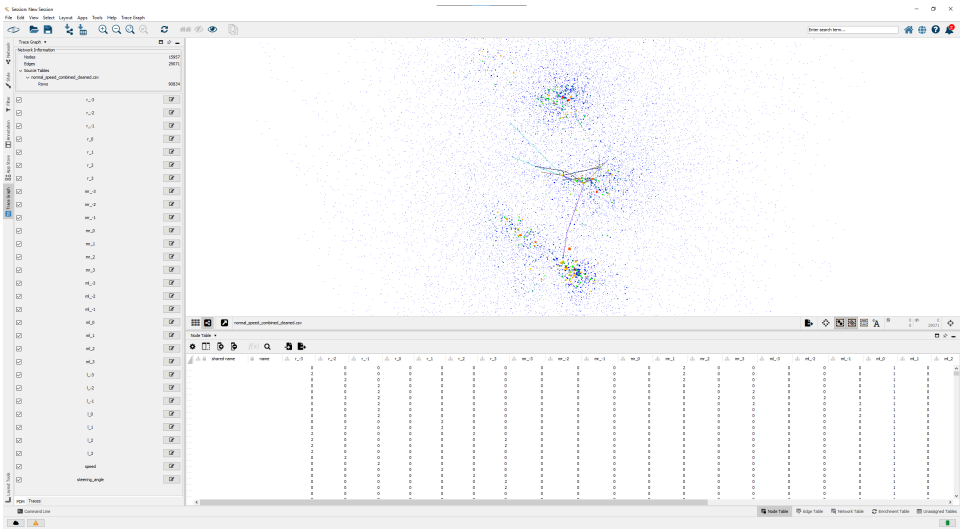

Figure 3.7: General layout of the application

Whenever a change is made to a Trace Graph, including editing a parameter's bin borders, adding a trace to the Trace Graph, merging or splitting Trace Graphs, etc., the graph is updated, the style is updated, and finally, the layout task is triggered. The update procedure is described in detail in section 3.3.7. The application uses an OpenCL-based version of the Prefuse force-directed layout algorithm. Prefuse is a set of general-purpose tools for visualizing information originally developed by Heer et al. [10]. Force-directed layout algorithms lay out graphs by simulating physical forces, treating nodes as repulsive charges and edges as springs, to create layouts with a low number of crossing edges.

3.3.3 Merging and Splitting

To merge or split Trace Graphs, the Cytoscape network panel must be selected. This shows the user all the currently available networks. They are grouped by their root networks, which Cytoscape refers to as *Network Collections*. If a single network is selected and it has more than one trace, it can be split. If two networks are selected, they can be merged. If the conditions for an action are not met, the menu item is grayed out. This is made possible by the *isReady* method of the *TaskFactory*. Before testing the above conditions, it checks if all selected networks are managed by the application. It does this by testing if a special marker column exists in the local network table, which is created when a network is created. If one of the networks is not managed by the application, the action is not allowed.

When the user chooses to split a Trace Graph, they are presented with a panel divided into two columns with all traces listed on one side. The user can then selectively move traces from one side to the other. Upon confirmation, the Trace Graph is split. A new Trace Graph is created, and the selected traces are removed from the old Trace Graph and added to the new one. Removing a trace from a Trace Graph involves filtering the nodes. If a node's traces, i.e., the set of all traces that have situations that map to the node, are a subset of the removed traces, then the node must be deleted along with its auxiliary information. Otherwise, the auxiliary information only needs to be corrected by removing the references to the removed traces. The node must not be removed because there is still at least one trace whose corresponding state sequence contains the state that corresponds to the node. The same is true for edges. An edge can be removed if all traces of the edge are included in the removed traces. This means that after the split, there would be no trace left whose state sequence contains a transition that corresponds to the edge.

After the nodes and edges have been filtered and references to the extracted traces have been removed from the auxiliary information of the remaining nodes and edges, two further steps are necessary. First, the *CyNode* arrays of the removed traces, which facilitate the mapping between situations and nodes, are removed from the Trace Graph. Second, the visit duration and frequency of nodes and edge traversals must be corrected since they may no longer be valid. The process of correcting auxiliary information is described in section 3.5.2.

Merging a Trace Graph A with another Trace Graph B works the opposite way. All traces from graph A are also added to graph B. No new nodes need to be created because the root network necessarily contains all the nodes from graph A. Graph B simply adds them to its own subnetwork. Then, graph A is deleted, freeing up all the resources of the Trace Graph. Deleting graph A does not delete any nodes from the root network because they are already present in graph B at that point. Finally, the style of graph B is updated to reflect the change.

Figure 3.8 shows a Trace Graph containing two traces, A and B, being split. Trace B is extracted. Each node is labeled with its frequency ("f"), visit duration ("vd"), and references to the traces containing its corresponding state. In reality, the auxiliary information contains not only the references to the traces but also the indices of the situations. The node with state 5 and its auxiliary information must be deleted because the state is not part of the state sequence of trace A. Nodes 2, 3, and 4 are part of the state sequence of both trace A and trace B. Therefore, they are not deleted, but the references to trace B are removed from their auxiliary information. In addition, their frequency values are now invalid and must be corrected. The only auxiliary information that remains valid is that for node 1. The graph now

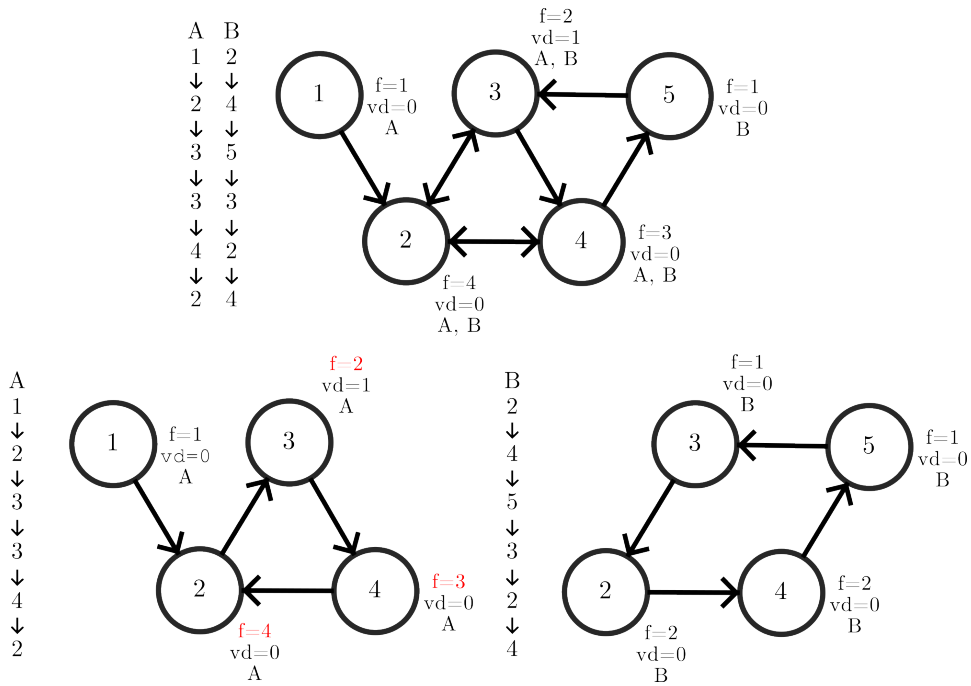

Figure 3.8: Splitting A Trace Graph containing two traces

containing trace B could be added back to the Trace Graph containing trace A, or vice versa, simply by adding the trace to the other Trace Graph and deleting the original Trace Graph.

3.3.4 Style and Mappings

By default, a node's size is determined by its frequency, and its color is determined by its visit duration. In most traces, certain types of situations and combinations of parameter values occur much more frequently than others. As a result, the states to which they are mapped will also appear more frequently in the state sequence, and the visit duration and frequency attributes will also be distributed unevenly. If the mapping between frequency and node size were linear, some nodes would be orders of magnitude larger than the majority of nodes, which would be almost unrecognizably small in comparison. Similarly, the differences in node colors of the nodes with the smallest visit duration values would be almost indistinguishable. A logarithmic mapping is used to alleviate this problem. The size mapping takes the minimum and maximum frequency values of a Trace Graph and a predefined minimum and maximum node size and maps the frequency values to that range using the natural logarithm as the factor for the mapping function. The minimum size is set to ten pixels and the maximum size is set to 100 pixels. Therefore, the node with the highest frequency will only be ten times the size of the node with the lowest frequency. Figure 3.9 shows the size mapping for a minimum node size of 10, a maximum node size of 100, and a hypothetical parameter value range of 0.1 to 1.

Since the auxiliary information of nodes and edges is not stored in the columns of a *CyTable*, the default implementations of *VisualMappingFunction*, which are accessible through a *VisualMappingFunctionFactory*, cannot be used. Instead, the size mapping is implemented as a passthrough mapping that receives a map of SUIDs to node frequencies on initialization. When the mapping receives a *CyRow*, instead of reading a value from the row and simply returning it, it takes the SUID of the *CyRow* and looks up the corresponding frequency in the input mapping. Rather than simply returning the value, as would be the intended use of a passthrough mapping, it passes the value to the size function, which then returns the final size of the node.

Similar to the size mapping, the color mapping first determines the minimum and maximum visit duration. Then, five color boundaries are placed along the range of values, which together mimic the *viridis* color scale. Purple is used for the lowest values and yellow for the highest. Blue is placed at 0.1% of the range, turquoise at 1%, and green at 10% of the range. If a node's visit duration does not perfectly

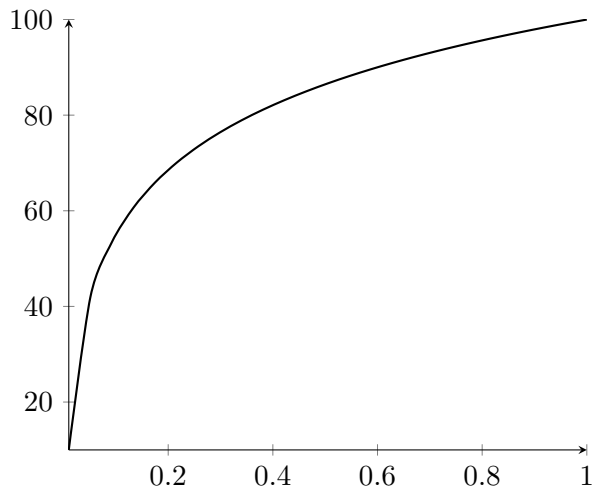

Figure 3.9: A Size mapping function

match one of the boundary values, the two closest boundaries are determined, and their colors are mixed. A weighted sum of the R, G, B, and alpha values of the lower and upper bounds is calculated depending on how close the value is to either boundary, resulting in the final color. The exponential scaling of color boundaries allows nodes with low visit durations to be distinguishable. Figure 3.10 shows the color gradient used for the mapping.

Because visit durations are also not stored as part of a *CyTable*'s column, the default *ContinuousMappingImpl*, which is strictly tied to a table column, cannot be used. Similar to the size mapping, the default *ContinuousMappingImpl* is adapted to receive as input a map from SUIDs to visit durations. The color mapping, which extends this customized continuous mapping, can then place colors at certain property values just as it would for the regular *ContinuousMappingImpl*.

A third mapping is used to provide the user with tooltips for nodes. When the user hovers over a node, it displays the parameters and the corresponding bin values of the node's state. A mapping is defined for the NODE_TOOLTIP visual property to facilitate this feature. It allows the user to view the state of a node without inspecting the table view that contains the node states. One disadvantage is that Cytoscape does not compute the tooltip only when a node is hovered but precomputes it for all nodes when the style is activated. This results in a slight memory overhead.

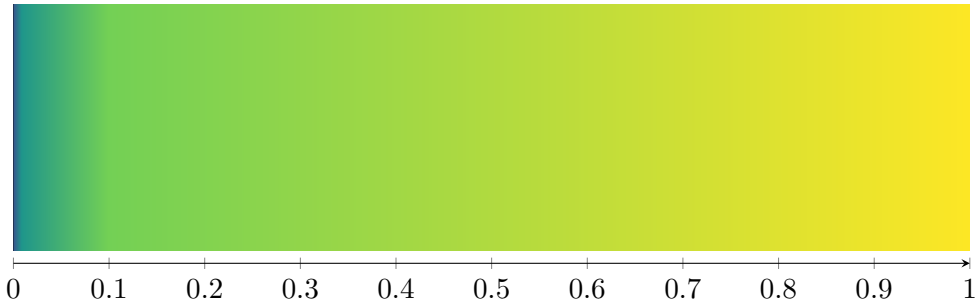

Figure 3.10: Color Mapping

3.3.5 Bin Modifications

The user can create, move, or delete the bin borders of parameters. This is done by clicking the edit button for any of the parameters listed in the default panel. A pop-up window will appear visualizing the frequency of the parameter values using a one-dimensional heat map constructed from a histogram of the parameter values. Since the distribution of parameter values in a trace is usually highly skewed, a Gaussian filter is applied by default to smooth the distribution and make it easier for the user to find hotspots. Bins can be moved by dragging a bin's triangle indicator above the heatmap, deleted by selecting the indicator and clicking delete, or added by double-clicking on a blank area. The new bin will appear at the location of the mouse click. Clicking the apply button applies the new bins to the parameter and leads to an update of all Trace Graphs that share the PDM for which the parameter borders are set. Figure 3.11 shows the bin customization dialog. Six bin borders have been defined, resulting in seven bins. In the center of each bin, the number of situations whose parameter values are within the range of the bin is displayed. The exact values of the bin borders are shown below the heatmap.

Alternatively, the user can enable or disable entire parameters by selecting or deselecting a parameter's checkbox in the default panel. This is equivalent to removing all bins for the parameter, as a parameter without bins has no effect on the transformation of traces into state sequences. Unchecking a parameter also hides it from the user in the tooltip that appears when the user hovers over a node and in the table view that lists the states of all nodes.

If the entire PDM needs to be updated rather than just a single parameter, the user can import a matching PDM file and then select the PDM to update. To improve performance, the PDM is marked as being updated instead of updating the bins one at a time. Any changes made to the PDM are then ignored by the *TraceGraphCon-*

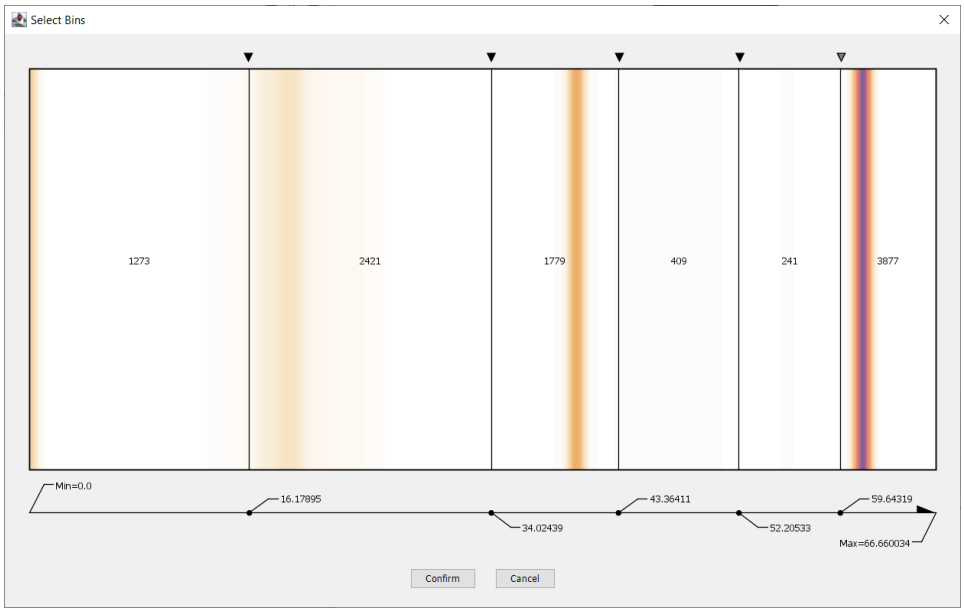

Figure 3.11: The bin customization dialog

trollers and the Trace Graphs are not automatically updated. Only once all bins have been set will an event be fired to which the *TraceGraphController*s subscribe, forcing the Trace Graphs to be updated. In this case, the PDM file does not require a *bins* field.

3.3.6 Hiding Nodes

By default, all nodes in the Trace Graph are visible. In the bin customization dialog, the user can right-click to select specific bins so that only nodes whose parameter bin value is the same as the selected bin are displayed. This can be done for multiple bins and multiple parameters simultaneously. Alternatively, the user can hide nodes based on frequency or visit duration percentiles. For example, if the user chooses to display the 20% frequency percentile, the nodes are ordered by frequency, and then the top n nodes are selected so that their sum of frequencies is equal to or less than 20% of the sum of all frequencies across all nodes. If nodes are hidden, a button appears in the default panel that, when clicked, will make all nodes visible.

Nodes are hidden by iterating over all nodes in the network. If a parameter contains selected bins, a node is only visible if its bin value matches that of one of the

selected bins. Otherwise, its *NODE_VISIBLE* property is set to false. This is done each time a Trace Graph is updated.

3.3.7 Update Procedure

Whenever the Trace Graph is updated, including loading traces, merging, splitting, modifying bins, replacing all parameter settings, but also changing the visibility of nodes, the *RenderingController* is called by the *TraceGraphController* that previously adjusted the Trace Graph to ensure that the view of the graph is updated accordingly. This is performed in four steps. First, the style is updated. If the auxiliary information changes, then the style mappings may need to be updated to reflect changes in the minimum and maximum values of traversals or visit durations. Then, the currently active *EdgeDisplayController* is notified. Its state may have become invalid due to changes in the graph. It also controls the style of the edges, which may also need to be updated. Next, nodes are hidden based on the filters selected by the user. Finally, the style is applied to the network and the layout task is performed. This must be done last because it only lays out the visible nodes.

3.4 Analysis

3.4.1 Rendering Modes

The user can choose between several rendering modes that determine which edges are displayed and what happens in response to edge selection events. Each mode has a corresponding Swing panel that displays helpful information or allows the user to configure the mode. There are three different modes the user can choose from. The first is the simplest of the three and displays only the outgoing edges of the currently selected node. When an edge is clicked, the adjacent node is selected and centered. This allows the user to follow a path formed by the edges.

The second mode displays all edges. Edges are colored differently depending on the number of times they are traversed. This allows the user to distinguish important edges from unimportant ones. The color mapping used for the edges uses a 4-color scale. Blue is used for the smallest values and red for the largest. Green is placed at 5% of the value range and orange at 10%. The color scale was chosen to be different from the one that is used for nodes so that the user can better distinguish between nodes and edges. Due to the large number of edges, the graph view can become confusing if all edges are displayed at the same time. Therefore, the configuration panel contains a slider that allows the user to set a threshold for the

minimum number of traversals an edge must have in order to be displayed. Due to the uneven distribution of edge traversals, most edges will only have a relatively small number of traversals.

The third mode highlights the paths of a certain length of all of the subtraces that contain situations that map to the selected node. These paths will have the selected node at their center. Given a length of n, for each continuous range of situations in a trace that maps to the state of the selected node, the next n states and the previous n states are determined. Again, each state can correspond to a single situation or to a range of situations if two or more subsequent situations map to the same state. Therefore, the state sequence of the subtrace contains $2n$ state transitions, but its total length may be greater because each state may correspond to a range of situations rather than a single one. The path of each of the resulting subtraces is displayed in a different color. If multiple paths contain the same edge, additional temporary edges are added so that the path of each subtrace is displayed independently. If a different node is selected or if the mode changes, the multiedges are removed so that there is only one edge connecting any two nodes.

When an important node is selected, the graph view can become confusing because too many edges are displayed at once. Therefore, the subtraces are assigned an importance score, and a slider in the corresponding panel allows the user to display only the top n most important subtraces. The importance of a subtrace is the sum of the frequency and visit duration of the states in the state sequence of the subtrace. An additional slider allows the user to select the length of the paths.

When an edge belonging to one of the paths is selected, a fourth mode is activated that is not directly accessible to the user and only displays the subtrace of the selected edge. This mode displays additional information about the subtrace in its panel. It is also activated when the user wants to find the shortest subtrace containing two or more nodes, which is presented in the next section.

3.4.2 Comparing Nodes and Finding the Minimal Subtrace

When the user selects two or more nodes simultaneously, a panel is displayed that compares the bin values of each parameter for all selected nodes in a table. The columns correspond to the nodes and the rows correspond to the parameter bin values. If a row has differing values, it is highlighted. To clarify which column corresponds to which of the selected nodes, the nodes are labeled and the header of each column contains the corresponding label. There is also a button that, when clicked, calculates the shortest subtrace that contains a situation from each of the selected nodes across all traces in the Trace Graph and displays its path. The algo-

rithm for finding the shortest subtrace is explained in section 3.5.3. The rendering mode for displaying the shortest subtrace also has a panel that lists all contained situations, the size of the subtrace, and the name of the trace to which it belongs. Clicking on a situation focuses the corresponding node in the graph view. This allows the user to follow the path formed by the subtrace.

3.4.3 Comparison

Selecting two Trace Graphs with the same PDM activates the menu item for comparing Trace Graph. The comparison allows the user to visually distinguish which nodes and edges belong to the base graph, the delta graph, or both. Base-only (BO) nodes and edges are colored green, delta-only (DO) nodes are colored red, and nodes in the base-delta (BD) subgraph are colored blue. The user can also optionally hide nodes or edges in the BO, DO, and BD sets using a 2x3 toggle grid. The *NetworkComparisonController* takes a snapshot of the node information of the base and delta graphs at initialization. The comparison is therefore unaffected by changes in the underlying networks. If the user wants to update a comparison, they must recreate it. This is necessary to ensure that the node information and the style of the comparison network do not become corrupted when one of the underlying Trace Graphs is edited.

The comparison graph is constructed by iterating over the nodes and edges of both graphs and adding them to the network. A column is created in the local node and edge tables to indicate the group membership of a node or edge. Discrete color mappings are created for both nodes and edges and linked to this column. An additional mapping is added for the node size. As with regular Trace Graphs, the size of a node is determined by its frequency. If a node is part of both Trace Graphs, the average of both frequencies is used instead.

3.5 Algorithms

3.5.1 Loading Trace Graphs

This section describes the algorithm for loading traces to Trace Graphs. Let $T = [t_1, t_2, \ldots, t_m]$ be a trace where t_i for $1 \leq i \leq m$ are the individual situations. Each situation is an n-sized array of parameter values $t_i = [t_{i1}, t_{i2}, ..., t_{in}]$. The algorithm for adding a trace to a Trace Graph, shown in algorithm 1, receives as input the trace T, the subnetwork S, the root network R, and the PDM. S must be a subnetwork of R and therefore contains a subset of the nodes and edges of R. The PDM can be viewed as an n-sized array of mapping functions.

Algorithm 1

```
ADDTRACE(𝒯, 𝒮, ℛ, 𝒫𝒟ℳ)
1:  let C = [1..n] be a new array
2:  prev ← nil
3:  node ← nil
4:  for i ← 0 to |T| − 1 do
5:      for j ← 0 to |PDM| − 1 do
6:          C[j] ← findBin(T[i][j], PDM[j])
7:      end for
8:      node ← FINDORCREATENODE(C, S, R)
9:
10:     node.situations[T] ∪ {i}
11:     if prev ≠ node then
12:         node.frequency ← node.frequency + 1
13:     else
14:         node.visitDuration ← node.visitDuration + 1
15:     end if
16:
17:     if prev ≠ nil and prev ≠ node then
18:         if (prev, node) ∉ S.E then
19:             S.E ∪ {(prev, node)}
20:             (prev, node).traversals ← 1
21:         else
22:             (prev, node).traversals ← (prev, node).traversals + 1
23:         end if
24:         (prev, node).situations[T] ∪ {i − 1}
25:     end if
26:     prev ← node
27: end for
```

Algorithm 2

FINDORCREATENODE($\mathcal{C}$, $\mathcal{S}$, $\mathcal{R}$)

1: $nodeID \leftarrow hash(C)$
2: **if** R contains a node with ID $nodeID$ **then**
3: $node \leftarrow R.getNode(nodeID)$
4: **if not** S contains a node with ID $nodeID$ **then**
5: $S.V \cup \{node\}$
6: $node.frequency \leftarrow 0$
7: $node.visitDuration \leftarrow 0$
8: **end if**
9: **else**
10: $node \leftarrow S.createNode()$
11: $node.id \leftarrow nodeID$
12: initialize node state with C
13: $node.frequency \leftarrow 0$
14: $node.visitDuration \leftarrow 0$
15: **end if**
16: **return** $node$

Line 1 defines an array $C = [c_1, c_2, ..., c_n]$ that will hold the state of a situation, i.e., each element of C is an integer bin value mapped from a parameter value using the PDM's mapping function for that parameter. Lines 2 and 3 define variables containing the nodes of the current and previous iteration steps. The iteration over the situations starts in line 4. First, the state of the current situation is computed by iterating over the parameters of the PDM and passing the parameter value $T[i][j]$ and the PDM's mapping function for the current parameter $PDM[j]$ to the $findBin$ method, and storing the result at index j of C. Then, the node for the calculated state is retrieved using the $findOrCreateNode$ subroutine, which is shown in algorithm 2.

$findOrCreateNode$ first calculates a hash for the state, which is used as a unique identifier for the node. This is a slight simplification, as in the implementation the hash is mapped to the node SUID, which is set by Cytoscape. If a node with the calculated $nodeID$ exists in the root network, it is retrieved. If it does not also exist in the subnetwork, it is added to it. At the same time, the auxiliary information for the node, which is specific for each individual subnetwork, is initialized by setting frequency and visit duration to 0. If the node does not exist in the root network, a new node is created in the subnetwork. This node is automatically also added to the root network. It is assigned the $nodeID$ and its state is initialized from C. In

the actual implementation, this translates to the node's row in the shared node table of the root network being populated with the state information. As before, visit duration and frequency are initialized. Then, the node is returned.

After the node has been added or retrieved in line 8 of $addTrace$, the auxiliary information for the node is corrected in lines 10 to 15. First, a reference to the situation i of trace T is stored for the node. Then, if the previous node is not equal to the node of the current iteration step, the frequency is increased by one. Otherwise, the node has remained the same and the visit duration is increased by one.

While lines 5 through 15 handle node creation and updates, the remaining lines focus on edges. If the node from the previous iteration step is different from the one that was created or retrieved in the current iteration step, an edge must be created if it does not already exist. Line 18 tests for the existence of this edge. If it does not exist, it is created and the edge traversals are initialized to 1. If it does, the edge traversals are simply incremented instead. In both cases, a reference to situation $i - 1$ of trace T is added for the edge. It would be equally possible to identify edges by the index of the current situation i rather than $i - 1$. Finally, at the end of each iteration step, $prev$ is set to the current node.

3.5.2 Updating Trace Graphs

When a bin border is added, moved, removed, or when a parameter is enabled or disabled, the Trace Graph must be updated to reflect the possible changes in the mapping between situations and states. A situation that was previously mapped to one state may now correspond to a different state and therefore a different node. Situations that need to be moved will be referred to as affected. Updating the Trace Graph requires both changes to the graph itself, i.e., deleting or adding nodes and edges, and corrections to the auxiliary information associated with nodes and edges. If a situation is mapped to a different node, the reference to the situation held by the old node must be moved to the new node. As a consequence, node frequency, node visit duration, and edge traversals must also be corrected. As mentioned before, a naive approach to ensuring the integrity of the Trace Graph would be to delete all nodes and edges, delete all auxiliary information associated with nodes and edges, and recalculate the entire graph based on the new parameter settings. However, this would be inefficient as the majority of situations are not affected by an update to the bins of a parameter.

When a bin border of a parameter is moved, only those situations for which the relevant parameter value maps to one of the two adjacent bins may be affected. For

example, if a bin border i, which separates bins i and $i + 1$ and is set at a value of k, is moved to a value of $k + d$ with there being no other bin borders between k and $k + d$, then only those situations whose parameter values lie between k and $k + d$ need to be moved. Their parameter value now maps to bin i instead of bin $i + 1$. Any other situation will still map to the same state.

When a bin border is added or removed, more situations are affected. If a bin border i that is placed at a parameter value of k is removed, all situations whose parameter value is greater than k must be moved. Their bin value for the changed parameter decreases by 1. If a bin border is inserted between the bin borders i and $i + 1$ at a value of k, the opposite happens. The bin values for the changed parameter of all situations whose parameter value is larger than k increase by 1.

Instead of recalculating the entire graph, the following algorithm moves situations affected by a changing parameter to the correct node, which may not yet exist. Unnecessary node creations and deletions are thus avoided. This results in a performance improvement, which is examined in section 4.2.2. If all nodes are deleted, the entire state for all nodes has to be recalculated. However, as only one parameter has changed, it is sufficient to test whether the bin value for the changed parameter is different to determine whether a situation needs to be moved. All remaining bin values of a situation are necessarily still valid.

The algorithm receives as input the index of the modified parameter p, the subnetwork S, the root network R, and the PDM. It also receives the set of traces U that make up the Trace Graph. Each trace is an array of situations. The nodes and edges present in S and R must correspond to the situations of the traces in U, as if the traces in U were added to the Trace Graph one by one using ADDTRACE.

First, all edges in the subnetwork are removed. They are recreated later, after all situations have been processed, instead of being adjusted as the algorithm progresses. Removing the edges also implies deleting all edge-related auxiliary information. Then, the algorithm starts to iterate over the traces U and the situations of each trace. j is the index of the current situation and T is the current trace. Next, information about the node associated with the situation is collected. This includes the node itself, the state of the node, and its bin value for the parameter with index p. In the implementation, this information is accessible through various data structures that track the state of the Trace Graph. More information about these supplementary data structures can be found in section 3.2.2.

Afterwards, the old bin value is compared to the new one. If they are equal, the situation does not need to be moved to another node and the next situation is processed. Otherwise, the bin value at the index of the changed parameter p of the old

Algorithm 3

```
UpdateTraceGraph(p, 𝒰, 𝒮, ℛ, 𝒫𝒟ℳ)
 1: S.E = {}
 2: for i ← 0 to |U| − 1 do
 3:     T ← U[i]
 4:     for j ← 0 to |T| − 1 do
 5:         oldNode ← findNode(T, j)
 6:         oldState ← oldNode.state
 7:         oldBin ← oldState[p]
 8:         newBin ← findBin(T[j][p], PDM[p])
 9:         if oldBin ≠ newBin then
10:             oldState[p] ← newBin
11:             node ← FindOrCreateNode(oldState, S, R)
12:             node.situations[T] ∪ {i}
13:             oldNode.situations[T] ← oldNode.situations[T] \ {i}
14:         end if
15:     end for
16: end for
17: GenerateEdges(𝒰, 𝒮, ℛ)
18: for each node ∈ S.V do
19:     if not there exists a trace T s.t. node.situations[T] ≠ {} then
20:         S.V ← S.V \ {node}
21:     end if
22: end for
23: FixAux(𝒮)
```

Algorithm 4

GENERATEEDGES($\mathcal{U}$, $\mathcal{S}$, $\mathcal{R}$)

```
node ← nil
for i ← 0 to |U| − 1 do
    prev ← nil
    T ← U[i]
    for j ← 0 to |T| − 1 do
        node ← findNode(T, j)
        if prev ≠ nil and prev ≠ node then
            if (prev, node) ∉ S.E then
                S.E ∪ {(prev, node)}
                (prev, node).traversals ← 1
            else
                (prev, node).traversals ← (prev, node).traversals + 1
            end if
            (prev, node).situations[T] ∪ {j − 1}
        end if
        prev ← node
    end for
end for
```

Algorithm 5

FIXAUX($\mathcal{S}$)

```
for each node ∈ S.V do
    node.frequency ← 0
    node.visitDuration ← 0
    for T ∈ node.situations do
        prev ← nil
        for i ∈ node.situations[T] do
            if i = prev + 1 then
                node.visitDuration ← node.visitDuration + 1
            else
                node.frequency ← node.frequency + 1
            end if
            prev ← i
        end for
    end for
end for
```

state is replaced with the new bin value. The node corresponding to the new state is retrieved with FINDORCREATENODE. This node is either a node that already exists in the root network and possibly the subnetwork or a newly created node. The current situation j is then assigned to the new node and removed from the old node. While it would be possible to fix the frequency and visit duration of the old and new node at this point, it is more efficient to fix them after all situations have been reassigned. Otherwise, the frequency and visit duration for a node may be unnecessarily overridden several times during execution.

Once all the situations have been processed, the algorithm must perform three additional tasks to restore the integrity of the graph. First, it must recreate the edges that were removed at the beginning of the algorithm. GENERATEEDGES performs this task in a way that is equivalent to how edges are created when a Trace Graph is loaded. It iterates over all traces and their situations, keeping references to the node of the current iteration and the node of the previous iteration step. If the two nodes are different and the edge does not already exist, it must be created and its auxiliary information initialized. Otherwise, the traversals must be incremented.

Lines 18 through 22 remove all nodes from the network that have no references to situations after the situations have been reassigned. As with the correction of traversals and visit durations, these nodes could also be removed during the reassignment. However, it is possible for a node to have no situations only temporarily. These nodes would be unnecessarily removed and added back later.

Finally, FIXAUX, which is shown in algorithm 5, restores the frequency and visit duration of all nodes. It does this by first resetting frequency and visit duration to 0 and then iterating over all traces and situations to which the node holds references. Two consecutive situation indices increase the visit duration by one, and if an index is not preceded by its numerical predecessor, the frequency is increased by one. This is similar to the way frequency and visit duration are initialized when a trace is loaded.

3.5.3 Finding the Minimal Subtrace

Let $A = [a_1, a_2, \ldots, a_m]$ be a state sequence where a_i for $1 \leq i \leq m$ are the individual states and a set of states $S = \{s_1, s_2, ..., s_n\}$. A could also be a sequence of nodes and S a set of nodes. Assuming that each state in S occurs at least once in the state sequence A, the minimum window $W = A[w_{min}, w_{max}]$ of A containing all states in S at least once can be determined in O(n) [1]. In the code, the algorithm has been slightly adapted to accept a trace as its input rather than a state or node sequence. The only change that is necessary is the resolution of the state or node

that a situation corresponds to. The problem is closely related to the "Minimum Window Substring" problem, which is about finding the minimum substring of a string A that contains all characters of a string B [13]. B can also contain duplicates. Therefore, the assumptions for this problem are slightly relaxed compared to the "Minimum Window Substring" problem since the elements in S are unique.

Algorithm 6

MINIMALSUBTRACE($\mathcal{A}$, $\mathcal{S}$)

1: $start \leftarrow 0$
2: $end \leftarrow |A| - 1$
3: let C be a new associative array
4: **for** $i \leftarrow 0$ to $|A| - 1$ **do**
5: **if** $A[i] \in S$ **then**
6: $C[A[i]] \leftarrow i$
7: **if** $|C| == |S|$ **and** $max(C) - min(C) < end - start$ **then**
8: $start = min(C)$
9: $end = max(C)$
10: **end if**
11: **end if**
12: **end for**
13: **return** $(start, end)$

The algorithm requires only one iteration over the state sequence. Before the iteration starts, two variables, $start$ and end, are defined, which contain the first and last index of the smallest window found at any point during the iteration. An associative array C is also defined, which maps states to an index of A. The largest and smallest indices must be efficiently retrievable. A possible data structure is outlined after the explanation of the algorithm. Line 5 tests whether the state at the current index i is an element of S. If it is, then the value for the state $A[i]$ in C is replaced with i. This means that the values in C will always correspond to the largest index found so far for any given state in S. The condition $|C| == |S|$ tests whether each state in S has been seen at least once. This assumes that C is dynamically resizable. If every state has been seen following an iteration i, then the subarray A[start, end] is the smallest subarray that contains all elements in S but only considering the elements in A[1,i].

In each of the following iteration steps, there is a chance that the state at the current index $A[i]$ is the state with the lowest value in C. If the recorded index for that state is replaced by i, the minimum value is removed from C and the maximum value in C becomes i. The new window $A[min(C), max(C)$ may be smaller than

the current smallest window $A[start, end]$ if the difference between the former smallest (removed) and second smallest (now the smallest) values in C is greater than the difference between i (now the largest) and the former largest value. Even if the recorded index for a state is not the minimum value in C, it must still be updated with i. Otherwise, the window may be too large once the state with the smallest index is found. Therefore, by repeatedly updating the index of the current state with i, it is guaranteed that after a full iteration over A, the smallest possible window is $A[start, end]$.

The smallest and largest elements in C can be obtained in O(1). C can be implemented as a combination of a linked list, which stores the indices, and an associative array, which maps from states to elements in the linked list. When the value for a state needs to be updated, the list element is looked up in the map and removed from the linked list in constant time. The new value for a state, i, is always greater than any element in the linked list and can simply be appended. Therefore, the list is always implicitly sorted, and the smallest element is always the first element. Java provides a *LinkedHashMap* type that guarantees a predictable iteration order based on the insertion order. Since each appended element is always the largest, the smallest element is also the first element returned by an iterator over the map. As a Trace Graph can contain multiple traces, the above algorithm must be applied to each trace individually, and the minimum of all traces is returned and displayed to the user.

3.5.4 Minimal Subtrace using ILP

An alternative method of computing the minimal subtrace, which defines the problem as an integer linear program (ILP), was also explored but was found to perform considerably worse than the solution presented in the previous section. It is related to the problem of limiting the number of start-ups of a machine in an industrial setting [12].

$$\min \sum_{i=1}^{n} x_i$$

$$\sum_{i \in situations(node)} x_i \geq 1 \text{ for } node \text{ in } nodes \tag{1}$$

$$s_i \geq x_i - x_{i-1} \text{ for } i \text{ in } 2, \ldots, n \tag{2}$$

$$x_1 \rightarrow s_1 \tag{3}$$

$$\sum_{i=1}^{n} s_i = 1 \tag{4}$$

$$s_i \in \{0, 1\} \text{ for } i \text{ in } 1, \ldots, n \tag{5}$$

$$x_i \in \{0, 1\} \text{ for } i \text{ in } 1, \ldots, n \tag{6}$$

Two boolean variables, x_i and s_i, are declared for each situation in the trace. If x_i is 1, then situation i is part of the minimal subtrace. In the optimal solution, there is a single continuous sequence of x_i that are set to 1. For each of the input nodes, there must be at least one index i such that the situation at index i maps to the node and x_i is 1. This is captured by constraint 1.

s_i indicates whether the situation at index i is the first situation in the subtrace. Constraint 4 guarantees that only one s_i can be set to one, so there is only one unique starting point. Constraint 2 ensures that the x_i form a sequence. If s_i is 1, then x_i must be 1, and x_{i-1} must be 0. Therefore, it is not possible for an x_i with an index less than the starting index to be set to 1. If s_i is 0, then x_i must be smaller or equal to x_{i-1}, i.e., either both x_i and $x_i - 1$ are 0, in which case neither is part of the smallest subtrace, both are 1, in which case they are both part of the smallest subtrace, or x_i is 0 and $x_i - 1$ is 1, which marks the end of the sequence. Constraint 3 ensures that the above conditions also hold for the first element in the state sequence. If x_1 is 1, then s_1 needs to be 1 to mark the start of the sequence. If this condition was missing, it would be possible that the sequence is split in two. A first sequence of x_i would start at index 0, despite s_1 being set to 0, and the actual, intended sequence would start at the index where s_i is 1. The objective function minimizes the sum of all x_i.

This approach was tested using the CP-SAT solver from Google's OR-Tools [2]. Regardless of the number of nodes that are selected, it can take several minutes to find the optimal solution. It therefore demonstrates significantly lower performance compared to the algorithm presented in the previous section. Due to the substantial differences in performance, the two approaches were not compared in more detail.

3.6 Misc

3.6.1 Ding Renderer Improvements

Due to Cytoscape's component-based architecture, the rendering engine can be customized for each network. *NetworkViewRenderers* can be registered with the *ApplicationManager* using a renderer ID. This can be used to create a *NetworkView-Factory*, which in turn provides a method to create a network view using a specific rendering engine. The rendering engine used by the application is a forked version of the default *Ding* rendering engine and has two enhancements. The first changes the default font of node tooltips to a monospaced font. This is a simple measure to allow tooltips to have two-column formatting, with one column containing parameter names and the other containing parameter bin values. Tabs can be used for adaptive spacing between names and values.

The second enhancement ensures that edges that cross the view area are always displayed. By default, edges are only displayed if one of their nodes is visible on the screen. This is sufficient for small graphs, or in the case of larger networks, it may be helpful to avoid a large number of edges crossing the screen and thus obscuring the edges of the visible nodes. However, it can become confusing if the network is large and the nodes are far apart, as is sometimes the case with Trace Graphs. Also, the number of edges visible on the screen can be controlled by the user. Therefore, it is more advantageous to show edges that only cross the screen than to hide them.

By default, Cytoscape renders edges by querying the nodes visible on the screen and retrieving their adjacent edges. The routine has been adapted to query all visible edges and to test for each edge whether the line formed by its adjacent nodes crosses the display area. Generally, this approach performs poorly when all edges are visible. However, since the number of edges visible on the screen is controllable by the user and therefore small in most cases, the solution is feasible and does not lead to significant performance degradation.

3.6.2 Custom CyTable

The default *CyTable* implementation used by Cytoscape is designed to use SUIDs as the primary key type for tables. Therefore, the key space for a table must be non-continuous and non-zero based. The number of rows and columns is also not known in advance because the user can create any number of additional rows or columns. To satisfy these conditions, the default implementation uses a hashmap to map from column keys to column values for each column separately. When the

key type is *Long*, which is the default for SUIDs, it uses a type-optimized hashmap to avoid having to store wrapper types as keys or values in a non-optimized map. The map implementation for *Double* value types uses the *fastutil Long2Double OpenHashMap*, which uses an open addressing approach with linear probing. It maintains a *long* and a *double* array, which it extends as necessary by rehashing all entries. If a table with a *Long* primary key type has n columns of value type *Double*, then n *long* and n *double* arrays are created. The key arrays are exact copies since all operations are always performed identically for each column.

The assumptions made for the default *CyTable* implementation do not apply when storing traces. Each trace has a fixed number of situations, which translate into a fixed number of rows and columns. The columns only need to store *doubles* and no arbitrary types. No new situations are added to or removed from a trace. Also, each situation is identified by its index instead of a random identifier. The application uses a more efficient *CyTable* implementation designed around these relaxed assumptions. It uses a hashmap from column names to double arrays. When a column is added to the table, the parameter name is mapped to a new array with a size equal to the number of situations in the trace. Rows can be easily retrieved using the situation's index. The improvements in terms of memory consumption are analyzed in section 4.3.2.

Chapter 4

Evaluation

The application is evaluated using a trace that consists of 30 parameters and contains 90834 situations. It was recorded in a virtual environment using the CARLA traffic simulator, where part of the drive was recorded on the freeway and part in an urban environment. When transformed into a Trace Graph using a PDM that was calculated using the method outlined in section 2.2.5, it contains 15957 nodes and 29071 edges. The average node has about 5.7 situations. The distribution is heavily skewed, however: 10016 nodes only have a single situation associated with them, while the top 100 nodes alone contain 43708 situations, which is roughly 48% of all situations. To compare how well the application performs for Trace Graphs of different sizes, 9 traces are generated from the original trace by pruning it to multiples of 10000. Thus, the first dataset contains 10000, the second 20000, and so on until the last, which contains 90000 situations. Figure 4.1 shows that the number of nodes and the number of edges scale approximately linearly with the size of the trace.

4.1 Methodology

The application is evaluated in two ways. First, the absolute performance is tested in terms of memory consumption and execution times for loading and updating Trace Graphs. Then, different approaches to the same tasks are evaluated. The tests are performed on a machine with a Ryzen 7 5700X 8-core processor at 3400 MHz and an AMD Radeon RX 6750 XT running Windows 10 version 22H2 build 19045.4046. The Cytoscape version is 3.10.1. Memory usage is analyzed using the Eclipse Memory Analyzer (MAT), and execution times are evaluated by averaging the results of running a task 20 times. The Intellij IDEA Profiler is used to analyze

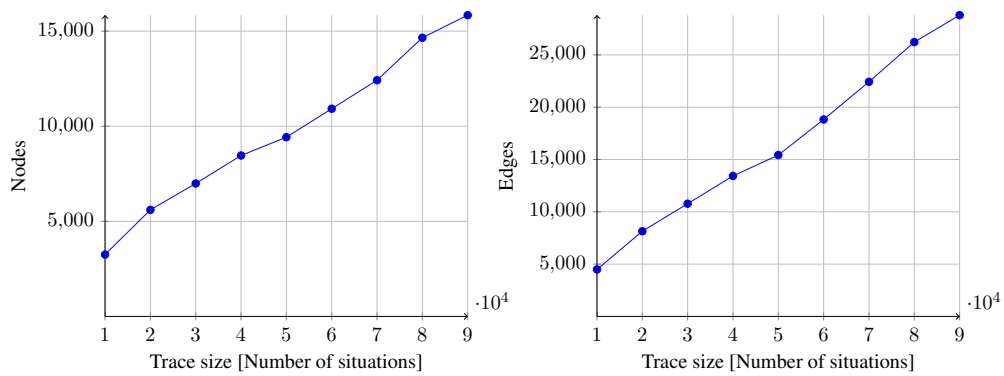

Figure 4.1: Number of nodes (left) and number of edges (right) per trace size

the performance in more detail. Two values are measured for each test. The first includes only the time it takes the algorithms themselves to complete and is referred to as the processing time. The second also includes the time Cytoscape takes to render the graph and perform various other side effects that may be triggered in response to model updates. It is the difference between the total execution time and the processing time and will be referred to as the rendering time. Despite its name, it also includes tasks that are not directly related to rendering the graph, such as updates to the search index.

Garbage collection is performed, and any outstanding Cytoscape events are flushed before each task execution to ensure a consistent testing environment. Additionally, there is sufficient wait time between tasks to avoid throttling issues, and the process is given a *RealTime* priority to avoid preemption as much as possible. The JVM is Eclipse Temurin 17.0.10+7. Due to JIT (just-in-time) compilation, the first tasks may be slower than the subsequent tasks. To avoid this problem, the measurement starts only after a number of initial dummy tasks have been executed, which are excluded from the result.

4.2 Individual Performance Assessment

4.2.1 Loading Traces

The execution times for loading traces are shown in figure 4.2. Both the total execution time and the processing time exhibit an approximately linear relationship with respect to the trace size and closely resemble the plot for the graph size per trace length. The processing time divided by the total running time is displayed on the right in figure 4.3. The share of the processing time increases from about 4%

for the smallest trace size to about 16% for a size of 50000 situations, from where on it stabilizes. The initial increase is mainly caused by tasks that have a relatively larger impact on execution times when the trace size is small compared to when it is large. For example, dynamically sized data structures, such as Java's *HashMap*, need to be resized if their initial size is not sufficient. When the trace size is small, the resizing can make up a significant portion of the overall execution time. If the trace is large, however, this cost gets amortized. Furthermore, dynamically sized data structures typically do not increase in size linearly but with a factor larger than one. This further reduces the impact on tests with larger traces. With an increasing graph size, the share that these tasks have on the total execution time becomes negligible and the ratio stabilizes. The remainder of the total execution time is spent registering the network view, applying the style, rendering the graph canvas, and performing the graph layout.

The rendering effort depends largely on the number of nodes rather than the size of the trace. Figure 4.3 on the left shows the average number of nodes rendered per millisecond, i.e., the number of nodes for different trace sizes divided by the rendering time. It quickly rises to a rate of approximately 4.6 nodes per millisecond from where on it is stable with a slight reduction to 4 nodes per millisecond for traces with a size of 50000 situations or larger. This rapid and consistent reduction may be due to garbage collection, which may have a more significant impact once traces reach a certain size. The initial increase may again be caused by tasks that get amortized for larger traces, such as the initial registration of the network view or resizings.

The constant share of the processing time compared to the total time does not necessarily exist for all Trace Graphs. The number of nodes does not necessarily grow approximately proportionally to the size of the trace. If a testing vehicle repeatedly encounters the same types of scenarios throughout the test drive, new situations may become less frequent over time. Therefore, fewer new states would be discovered for the later situations of a trace, and fewer new nodes would be created for situations that are processed later. Because the rendering effort scales with the size of the graph and the algorithm scales with the size of the trace, the share of the processing time would become more dominant as the trace grows larger.

4.2.2 Updating Trace Graphs

This section analyzes the execution time for Trace Graph updates using the algorithm presented in section 3.5.2. It examines the impact of the number of affected situations on performance. As explained in section 3.5.2, if a bin border that is placed at a value of k is deleted, and k is small compared to the parameter value

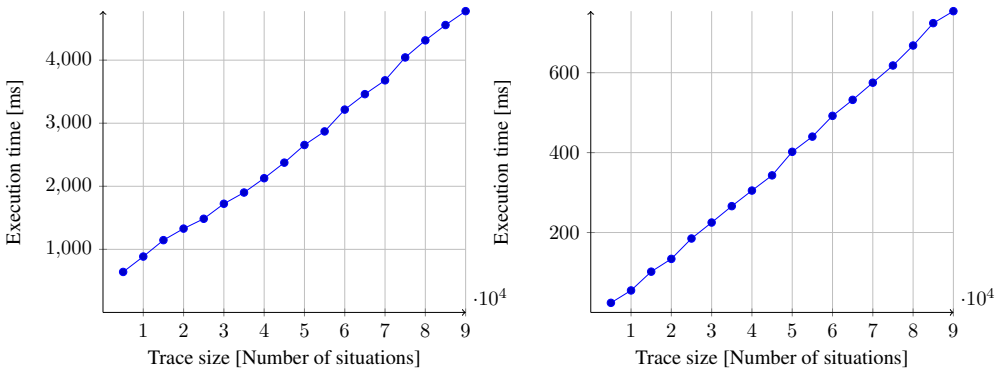

Figure 4.2: Total execution time (left) and processing time (right) for loading traces

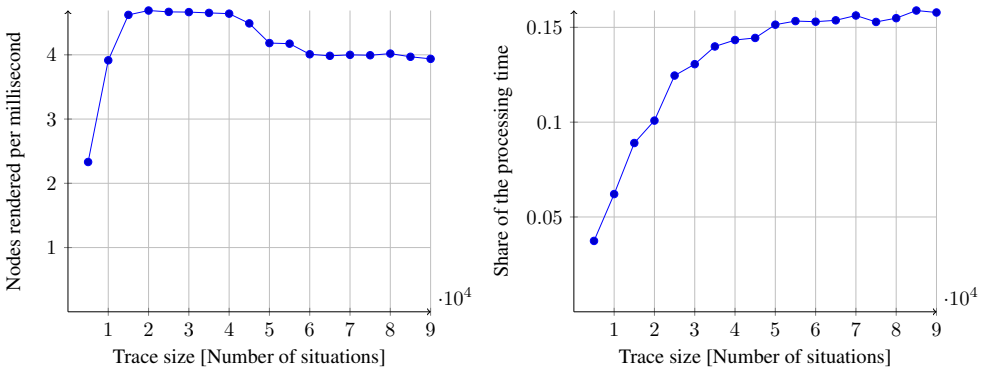

Figure 4.3: Number of nodes rendered per millisecond (left) and share of the processing time on the total execution time (right)

range, it will take more effort to restore the integrity of the Trace Graph than if k is large since all situations with a parameter value greater than k must be moved from one node to another. The same is true if a bin border is inserted at a low compared to a high value in the parameter value range. To validate this assumption, the execution times for placing bin borders at 12 different positions in the speed parameter value range are tested. The positions of the new bin borders are chosen so that the difference in the number of affected situations between two successive bin borders is approximately constant. This means that the first placed bin border will affect all 90834 situations, and each subsequent border will affect approximately $90834/12 = 7569$ situations less than the previous border. The difference will not always exactly match this number as the values are not evenly distributed. The effort required to update a Trace Graph after moving a bin border is not examined here but is, on average, considerably lower than for bin border insertions or dele-

tions. When a bin border is moved, only a fraction of the situations in the adjacent bins need to be moved.

There are several side effects that are triggered in response to Trace Graph updates. Besides applying the style, performing the graph layout, and rendering the graph, other side effects include updates to the Cytoscape search index, which was already explained in section 3.2.2, and the caching of the attributes of deleted nodes and edges. The caching allows nodes and edges that have been deleted in all subnetworks of a root network to be restored. It is primarily used by the Undo support provided by Cytoscape. This adds an overhead to the initial deletion of all edges and the final deletion of nodes that need to be deleted because they no longer have situations associated with them. While the search and caching features are not necessarily helpful to the user of the application, it is not possible to disable them without changes to Cytoscape itself.

Figure 4.4 shows that the total execution time and the processing time are approximately linear with respect to the number of affected situations. The rendering time is constant, taking on average 1794 ms with a standard deviation of 55 ms. This is because the size of the graph is approximately equal for all runs, with an average of 16705 nodes at a standard deviation of 500 nodes. Adjusting for the graph size by dividing the rendering time by the number of nodes in each run yields approximately 9.08 nodes rendered per millisecond, with a standard deviation of only 0.12. Thus, the linear component of the total execution time can be fully explained by the processing time. While the processing time scales linearly with the number of affected situations, the offset is not 0 but about 300 ms. This is because the algorithm requires the removal and recreation of edges. The number of edges to be removed and regenerated is approximately constant for all graph sizes.

The proportion of the processing time to the total execution time increases with the number of affected situations. This is shown in figure 4.5. It increases from about 18.5% for the smallest number of affected situations to 43% when all situations are affected. While the rendering time is constant, the processing time scales with the number of affected situations. This trend is expected to continue for even larger updates.

4.2.3 Memory Usage

Heap Dumps are acquired and analyzed using the Eclipse Memory Analyzer (MAT). The memory consumption can be divided into two categories. The first relates to the amount of memory that is necessary to store information about the Trace Graph itself. This includes the traces, nodes, and auxiliary information but also additional

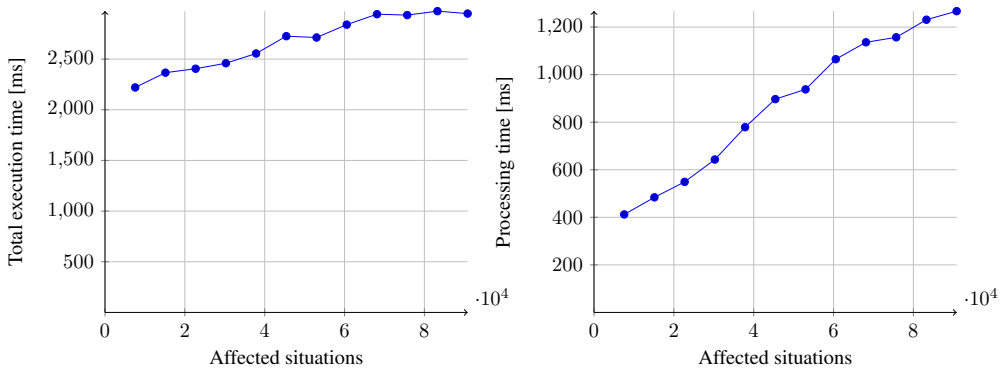

Figure 4.4: Total execution time (left) and processing time (right) of Trace Graph updates

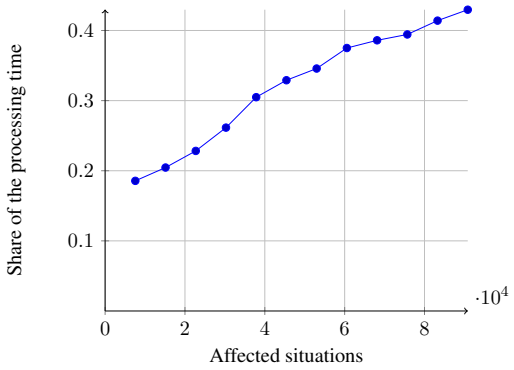

Figure 4.5: Share of the processing time on the total execution time

data structures such as the mapping from state hashes to SUIDs or the mappings from situations to nodes. The second category contains objects that are necessary to render the graph but that are still tied to the Trace Graph, such as the visual properties and style mappings.

Memory consumption is measured using the retained heap size calculated by the MAT. It is the amount of memory that is held by an object and those transitive references that are only accessible through references to said object. Objects that are part of the retained set of an object A are eligible for garbage collection if A is garbage collected. Some objects have to be excluded from the calculation of the retained heap sizes in order for the measurements to correctly reflect memory consumption. For example, Cytoscape renders a graph by creating a snapshot of the network view, which holds references to the network view's attributes. If

Object	%	% of parent	Size
Root network	5.08%		4158.88 kB
Subnetwork	5.08%		4158.50 kB
Trace	33.27%		27256.70 kB
Trace Graph	20.02%		16401.15 kB
Node Auxiliary Information		44.73%	7336.83 kB
Edge Auxiliary Information		53.05%	8700.56 kB
Situation-to-Node mapping		2.22%	363.52 kB
Tables	32.38%		26528.51 kB
Shared node table		50.17%	13308.15 kB
Shared edge table		10.25%	2718.01 kB
Local node table		6.71%	1780.37 kB
Local edge table		12.23%	3243.98 kB
Facades		20.65%	5478.00 kB
PDM	1.25%		1024.73 kB
Hash-to-suid mapping		100.00%	1024.73 kB
Nodes and Edges	2.92%		2393.91 kB
Nodes		32.00%	765.94 kB
Edges		68.00%	1627.98 kB
Total			81.92 MB

Table 4.1: Memory consumption of model related components

the snapshot objects were not excluded from the calculation, the network view's retained size would be underestimated since it would exclude the size of all attributes which are shared by the snapshot. By excluding the snapshot, which MAT allows by defining exclusion patterns, the retained size reflects the true memory consumption more closely. Another example is related to tables and table facades. Both the actual *CyTableImpl* and the *LocalTableFacade*, which is the type backing default tables, hold references to *CyTable$InternalRow*. To associate the size of table rows with the *CyTableImpl*, *LocalTableFacade* has to be excluded. Care has been taken not to exclude any large objects from the calculation of the total size. However, some smaller objects, which are negligible compared to the overall size, may not be included in the calculation.

Table 4.1 shows the memory consumption of those objects that store information about the Trace Graph itself. The total amount of memory is about 81.92 MB. Out of those 81.92 MB, 33.27%, or 27256.70 kB, is utilized storing the trace. As explained in section 3.6.2, the trace is stored as a hashmap that contains one *double* array per parameter. The trace has 30 parameters and 90834 situations. A *double* in

Object	%	% of parent	Size
CyNetworkViewImpl	56.51%		39987.42 kB
Node Visual Properties		70.23%	28084.13 kB
Edge Visual Properties		12.52%	5008.18 kB
Adjacent edge map		9.18%	3671.37 kB
Table Views	37.03%		26202.20 kB
Default Node Table		9.63%	2524.10 kB
Default Edge Table		18.05%	4729.94 kB
Trace Table		72.32%	18948.16 kB
Visual Style	1.82%		1285.22 kB
Color Mapping		50.02%	642.89 kB
Size Mapping		49.98%	642.34 kB
Swing Panel	0.13%		95.08 kB
Views	2.69%		1906.03 kB
Node Views		26.79%	510.62 kB
Edge Views		73.21%	1395.41 kB
Total			70.76 MB

Table 4.2: Memory consumption of components related to the rendering of the graph

Java is 8 bytes, and the minimum possible memory required is $8\ \text{B} \times 30 \times 90834 = 21800.16$ kB. Therefore, there is an overhead of about 25%, which is due to the table needing to maintain one persistent row object per row.

The main components of the *TraceGraph* are the node auxiliary information, the edge auxiliary information, and the situation-to-node mapping. Node and edge auxiliary information combined make up about 97.78% of the size of the Trace Graph. The root network and the subnetwork both utilize the same amount of memory at approximately 4.16 MB because both networks need to store the same number of node and edge pointers. The node and edge objects themselves have a size of only about 2.4 MB. The size of the PDM is almost entirely due to the hash-to-suid mapping, which is shared by all Trace Graphs. It is therefore not actually part of the retained set but is still included here. In comparison, the size of the parameters is negligible. The remaining memory is taken up by the tables. The shared node table requires about half of all table-related memory as it stores all state information. Even though the shared edge table, local node table, and local edge table do not store any information related to Trace Graphs, they still maintain a row object for each table entry. The remaining table-related memory is consumed by the table facades, which are the objects that facilitate default tables.

The main objects that store information that is necessary to render the graph are depicted in table 4.2. Their total size is about 70.76 MB. Among these objects, the network view is the larges, with a size of about 40 MB. It stores the visual properties that have been set for nodes and edges and some supplementary data structures that aid in rendering the graph, such as a map that stores edge views that are adjacent to a node. A large share of the node visual properties are the tooltips, which are generated by the tooltip mapping that is registered with the style. Cytoscape does not allow deferring the generation of tooltips until a node is hovered over. A Hash array mapped trie (HAMT) is utilized for storing visual properties.

The table views create one row view object per table row, even if rows are not explicitly styled. One table view is generated for the default node, default edge, and default network table, respectively. An additional table view is generated for the table that holds the parameter values of the trace. The visual style contains the visual mapping functions. Their size is due to a mapping from SUIDs to either frequency or visit duration, which are required to calculate the correct property value for each node. If the application used local or hidden tables for storing auxiliary information, the mappings could refer directly to the table columns instead of requiring additional mappings and their size would become negligible. The Swing panels add an additional 95 kB to the overall size. Node and edge views combined require about 1906.03 kB of memory.

The total size of the memory snapshot is 247.6 MB. Both types of Trace Graph related memory consumption combined require 152.68 MB of memory. The application itself, without loading a Trace Graph or performing any other action, takes up approximately 82.6 MB. This leaves only about 12.32 MB of memory unaccounted for, which may be due to memory required for the search index or due to various small objects that were not captured in the analysis.

4.2.4 Identification of Memory Leaks

A memory leak was discovered in the Cytoscape codebase that would cause all Trace Graph related data to remain in memory after deletion due to styles not being unregistered after being deleted. When a style is first created, it is registered as a listener for events that are triggered when visual mapping functions or visual property dependencies are changed. However, these registrations are not deleted when the visual style itself is removed from the *NetworkVisualMappingManager*, which is responsible for maintaining styles and mapping styles to networks. Therefore, the OSGi service registry still retains references to the styles, and they never become eligible for garbage collection. In this case, the tooltip mapping that cre-

ates the text for node tooltips had a reference to the PDM. As a result, the root network, hash-to-suid mapping, and parameters could not be cleared either, resulting in a gradual increase in system memory usage over time. Since the visual style registrations cannot be cleared by an external bundle without the use of reflection, a workaround was implemented to allow the tooltip mapping to work without reference to the PDM. The memory leak still exists but is much less consequential. The Cytoscape maintainers have been informed of this problem.

Another issue related to the deletion of nodes and edges was discovered, which also causes a gradual buildup of memory over time. When nodes or edges are removed from a subnetwork, they are also removed from the root network if they do not exist in any other subnetwork. In this process, entries for the deleted nodes and edges are temporarily created in the default node and default edge tables of the root network. These are equivalent to the default node and edge tables of a subnetwork but are not intended to be used actively. The created rows in the default tables would usually not be problematic as all rows for all associated tables of the deleted nodes and edges are subsequently removed. This includes the shared tables, as well as the default tables, and their underlying local tables. A problem arises if the rows are first cleared in a local table and then in the corresponding default table. When a row is generated for a default table, it is added to the underlying local table, and then a *RowFacade* is created, which wraps the row of the local table. This facade is stored in a hashmap that maps the actual row to the facade row. When the rows are cleared first in the local table and later in the default table, the facade rows will not be deleted because the default table requires the deleted rows to be present in the local table. Therefore, the facade rows remain in memory.

As a consequence, the actual rows are not eligible for garbage collection. However, the cache for deleted nodes and edges uses a *WeakHashMap* that maps from the actual rows to the node or edge properties. If there are strong references to the rows, the cache will also not be cleared. This problem was solved by clearing the row facades of the default tables using reflection after each update to the Trace Graph.

4.3 Relative Performance Evaluation

This section analyzes how different approaches to the same tasks compare to each other. First, the naive Trace Graph Update approach is compared to the more efficient approach. Then, the memory required by the custom *CyTable* implementation is compared to the default implementation.

4.3.1 Trace Graph Updates

This section compares the naive approach to updating Trace Graphs, which consists of removing all nodes and edges and regenerating the entire graph, with the more efficient variant that was already evaluated in section 4.2.2. The effort for updating the graph by regenerating it is not equivalent to that of loading a Trace Graph because deleting nodes and edges also requires computational effort. To compare the two approaches, the same setup is used as in section 4.2.2, this time using the naive update method.

As figure 4.4 shows, the running time of the efficient update approach scales with the number of affected situations. On the other hand, the naive approach is constant irrespective of the type of update that is performed. It does not matter whether one or multiple bin borders are moved, added, or removed or how many situations are affected because the graph is always completely rebuilt. On average, updating the Trace Graph using the naive variant takes 3022 ms with a standard deviation of 63 ms. Figure 4.6 shows the execution times of both approaches in a single plot. When all situations need to be moved, the naive approach performs almost identically to the efficient one because 84% (13364 out of 15957 states) of the updated states do not have nodes that already exist, and therefore, a large share of nodes need to be created and removed. For updates with a smaller number of affected situations, there are slight performance improvements. However, the overall execution time is significantly affected by the rendering overhead. Even when the number of affected situations approaches zero, resulting in minimal processing time, the performance improvement of the efficient variant over the naive version is only about 26.5%.

4.3.2 Custom CyTable Implementation

This section analyzes the efficiency gains of the array-based *CyTable* implementation presented in section 3.6.2 over the default *CyTable* implementation used by Cytoscape. Memory consumption is measured by importing the same trace that was used throughout the evaluation, which is 90834 situations long and consists of 30 parameters. Section 4.2.3 showed that the custom *CyTable* implementation requires 27256.70 kB, which is a 25% overhead over the theoretical minimum. The default implementation has a retained size of 73 MB and is therefore about 2.68 times larger.

Each column in the default *CyTable* is represented as a separate hash table that is based on two arrays. One array stores keys, and the other stores values. In this case, the keys are *long*, and the values are *double*. Approximately 31 MB is used for the 30 *long* arrays that store the keys. Another 31 MB is used for the *double* array

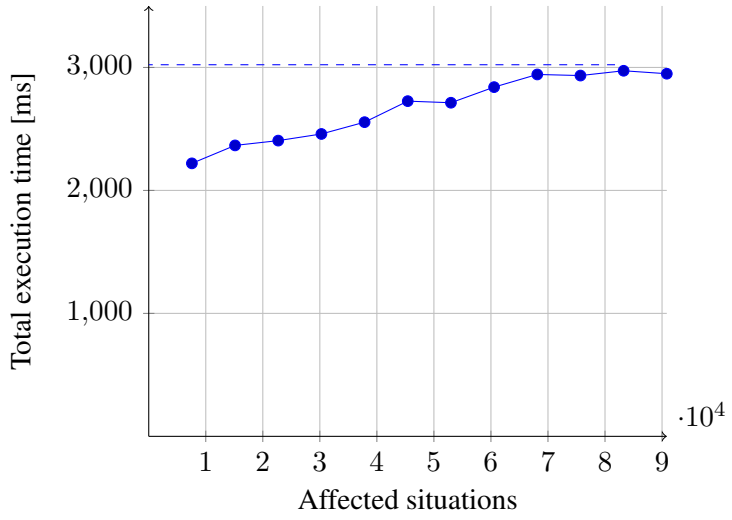

Figure 4.6: Total execution time using the efficient variant (solid) by trace size compared to the average execution time of the naive approach (dashed)

that stores the values. The remaining additional memory is due to the increased complexity of the default table implementation, which also has to support virtual columns.

There are two main ways in which the custom *CyTable* requires less memory compared to the default implementation. Each of the arrays in the default implementation has a size of 131073 due to the dynamic resizing of the hashmaps. Because the size of the trace is known upfront, the custom implementation does not need to support resizing and has no empty slots. Additionally, keys do not have to be stored explicitly in the custom implementation as situations can be accessed by their index.

4.4 Discussion

The evaluation shows that rendering large graphs using Cytoscape imposes a significant overhead for both running times and memory consumption. When loading traces, only about 16% of the total running time is spent initializing the Trace Graph, while the remainder is spent rendering the graph. The same holds true for updates to an existing Trace Graph, where about 18.5% for the smallest tested number of affected situations up to 43% for the largest number of situations is required for updating the graph. This means that possible improvements to the algorithms

would only yield a limited benefit to the user. It would have a far bigger impact to improve Cytoscape's internal components to better handle large graphs. Possible areas for improvement are outlined in the next chapter.

Despite the large overhead, the application maintains enough efficiency to be effectively usable. Loading the largest trace that was tested only takes 4.8 seconds on average. The loading times for even larger traces are expected to scale less than linearly because new states would become less frequent over time, especially for very large traces. Updating the same Trace Graph requires at most 3 seconds, even if all of the approximately 90000 situations need to be relocated. For simple updates, like when bin borders are moved, the update time is negligible. The full Trace Graph, including the trace, the state information, styling, and auxiliary information, requires only 152.68 MB of memory in total. Overall, there is no single drawback that would result in a significant deterioration of the user experience.

Chapter 5

Future Work

There are several areas that could be improved. One topic that has not been explored, but could yield significant performance improvements, is parallelization, from which both loading and updating Trace Graphs could benefit. The algorithms for loading and updating Trace Graphs iterate over the situations of all traces. It would therefore be possible to partition the situations and assign each partition to a different worker thread. When updating a Trace Graph with multiple traces, it would also be possible for each trace to be processed by a different thread. Cytoscape's data types are already thread-safe. However, changes to the algorithms would be necessary as they generally assume that situations are processed in order.

Updating Trace Graphs could also benefit from changing the data format of parameter bins. Currently, bins are stored as lists of type *double*. When bin borders are inserted or deleted, a large number of situations are affected because the bin value is incremented or decremented for all situations with a value larger than that of the inserted or removed bin border. This number could possibly be reduced by storing bin borders not as lists of type *double* but as sorted lists of ranges. The node state information would no longer consist of bin indices but of references to the range elements. When a bin border is inserted, a range object needs to be split, and when a bin border is removed, two ranges would be merged. Only the situations that point to the affected ranges would need to be moved. Therefore, the overall effort for updating the Trace Graph would be greatly reduced. Of course, this would require major updates to the application. The standard *CyTable* implementation cannot hold references to arbitrary types. Therefore, an indirection layer would have to be added to map the table entries to range references. As a result, the table view in the Cytoscape GUI that displays node states would no longer be helpful to

the user. Changes would have to be made to the Cytoscape codebase itself to add a table renderer that can handle the above changes and still display the table entries as bin indices.

The Trace Graph comparison feature could also be further enhanced. Currently, it only displays a union of the base and delta graphs and visually highlights which nodes belong to which of the three subgraphs. A method for selecting a representative set of traces to satisfy node coverage or other types of coverage criteria, as explained in section 2.2.4, could be implemented. Additionally, normal states and scenarios could be identified and highlighted using the method presented by Brinkmann et al. [4].

Given the large overhead imposed by Cytoscape, which was discussed in section 4.4, it may also be reasonable to improve those Cytoscape components that have the most significant impact on performance. For example, disabling the search and undo features would reduce the time it takes to update a Trace Graph. This is not possible using the Cytoscape API alone but would require changes to Cytoscape itself. Without the overhead imposed by the search index, it would also be possible to store auxiliary information in the local or hidden node and edge tables of the subnetworks. This would also allow the use of regular style mappings, which could be linked directly to the table columns, rather than the custom mappings now used that are specialized for the custom formats of node and edge auxiliary information. Additionally, changes could be made to only compute tooltips when a node is hovered instead of precomputing all tooltips when the style is registered.

Chapter 6

Conclusion

Overall, the application presented in this thesis meets all of the requirements listed in section 3.1. The user has the ability to import either a complete PDM or individual traces. They can explore and analyze the resulting Trace Graphs using features such as different rendering modes, mappings that highlight important nodes and edges, filters, or context-sensitive information panels. The steps involved in transforming traces to state sequences and finally to nodes in the Trace Graph are made transparent. When a node is selected, the user can view the corresponding state and is provided with the situations that map to the node. Additionally, two Trace Graphs can be compared to identify the base-only, base-delta, and delta-only subgraphs.

The Cytoscape platform proved to be a viable basis for the tool. In particular, its core data format, with the nesting of networks and the tight coupling between networks and supplementary data in the form of tables, allowed the design of an application that could clarify the transformation process from traces to Trace Graphs. In addition, its modular design allowed for more complex customizations, such as changes to the rendering engine. Despite these advantages, it did present some challenges. Features, such as the search index, showed limitations when applied to larger networks and required some workarounds and design changes. Overall, however, these issues did not significantly affect development or prevent the addition of certain features.

Bibliography

[1] Algorithm for finding the minimum window width in a string that contains all characters of another string. https://stackoverflow.com/questions/3592079/minimum-window-width-in-string-x-that-contains-all-characters-of-string-y [Accessed: 13.03.2024].

[2] Google or-tools. https://github.com/google/or-tools [Accessed: 13.03.2024].

[3] Gerrit Bagschik, Till Menzel, and Markus Maurer. Ontology based scene creation for the development of automated vehicles. In *2018 IEEE Intelligent Vehicles Symposium (IV)*, pages 1813–1820. IEEE, 2018.

[4] Jannik Brinkmann, Noah Metzger, and Christian Bartelt. Unsupervised extraction of test scenarios from time-series sensor data using trace graphs. In *Proceedings of the 57th Hawaii International Conference on System Sciences*, 2024.

[5] Jinkang Cai, Weiwen Deng, Haoran Guang, Ying Wang, Jiangkun Li, and Juan Ding. A survey on data-driven scenario generation for automated vehicle testing. *Machines*, 10(11):1101, 2022.

[6] Ching-Yao Chan. Advancements, prospects, and impacts of automated driving systems. *International journal of transportation science and technology*, 6(3):208–216, 2017.

[7] Riccardo Donà and Biagio Ciuffo. Virtual testing of automated driving systems. a survey on validation methods. *IEEE Access*, 10:24349–24367, 2022.

[8] Ecipse Foundation. Osgi governance transfer press release, 2021. https://newsroom.eclipse.org/news/announcements/osgi-

alliance-transfers-governance-osgi-specifications-open-source-software [Accessed: 01.03.2024].

[9] Daniel J Fremont, Edward Kim, Yash Vardhan Pant, Sanjit A Seshia, Atul Acharya, Xantha Bruso, Paul Wells, Steve Lemke, Qiang Lu, and Shalin Mehta. Formal scenario-based testing of autonomous vehicles: From simulation to the real world. In *23rd International Conference on Intelligent Transportation Systems (ITSC)*, pages 1–8. IEEE, 2020.

[10] Jeffrey Heer, Stuart Card, and James Landay. Prefuse: A toolkit for interactive information visualization. pages 421–430, 04 2005.

[11] Nidhi Kalra and Susan M Paddock. Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability? *Transportation Research Part A: Policy and Practice*, 94:182–193, 2016.

[12] Erwin Kalvelagen. Modeling number of machine start-ups, 2014. http://yetanothermathprogrammingconsultant.blogspot.com/2014/12/modeling-number-of-machine-start-ups.html [Accessed: 26.03.2024].

[13] Leetcode. Minimum window substring problem. https://leetcode.com/problems/minimum-window-substring [Accessed: 26.03.2024].

[14] Till Menzel, Gerrit Bagschik, and Markus Maurer. Scenarios for development, test and validation of automated vehicles. In *2018 IEEE Intelligent Vehicles Symposium (IV)*, pages 1821–1827. IEEE, 2018.

[15] Noah Metzger, Lars Hoffmann, Christian Bartelt, Heiner Stuckenschmidt, Michael Wommer, and Maria Belen Bescos del Castillo. Towards tracegraphs for data-driven test case mining in the domain of automated driving. In *2021 IEEE International Conference on Artificial Intelligence Testing (AITest)*, pages 41–48. IEEE, 2021.

[16] Demin Nalic, Tomislav Mihalj, Maximilian Bäumler, Matthias Lehmann, Arno Eichberger, and Stefan Bernsteiner. Scenario based testing of automated driving systems: A literature survey. In *FISITA web Congress*, 2020.

[17] Christian Neurohr, Lukas Westhofen, Tabea Henning, Thies de Graaff, Eike Möhlmann, and Eckard Böde. Fundamental considerations around scenario-based testing for automated driving. In *2020 IEEE intelligent vehicles symposium (IV)*, pages 121–127. IEEE, 2020.

[18] OSGi Working Group OSGi Alliance. Osgi core release 8 specification, 2020. https://docs.osgi.org/specification/osgi.core/8.0.0/ [Accessed: 01.03.2024].

[19] Paul Shannon, Andrew Markiel, Owen Ozier, Nitin S Baliga, Jonathan T Wang, Daniel Ramage, Nada Amin, Benno Schwikowski, and Trey Ideker. Cytoscape: a software environment for integrated models of biomolecular interaction networks. *Genome research*, 13:2498–2504, 2003.

[20] Cytoscape Core Team. Cytoscape developer documentation. https://cytoscape.org/documentation_developers.html [Accessed: 27.03.2024].

[21] Simon Ulbrich, Till Menzel, Andreas Reschka, Fabian Schuldt, and Markus Maurer. Defining and substantiating the terms scene, situation, and scenario for automated driving. In *2015 IEEE 18th international conference on intelligent transportation systems*, pages 982–988. IEEE, 2015.

[22] Lukas Westhofen, Christian Neurohr, Tjark Koopmann, Martin Butz, Barbara Schütt, Fabian Utesch, Birte Neurohr, Christian Gutenkunst, and Eckard Böde. Criticality metrics for automated driving: A review and suitability analysis of the state of the art. *Archives of Computational Methods in Engineering*, 30:1–35, 2023.

[23] Ziyuan Zhong, Yun Tang, Yuan Zhou, Vania de Oliveira Neves, Yang Liu, and Baishakhi Ray. A survey on scenario-based testing for automated driving systems in high-fidelity simulation, 2021.

The source code, documentation, and additional test results are available at `https://github.com/felixkroemer/trace-graph-engineering-tool`.

They, as well as a PDF version of this thesis, are also contained on the USB flash drive attached to this thesis.

Ehrenwörtliche Erklärung

Hiermit versichere ich, dass diese Arbeit von mir persönlich verfasst wurde und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn diese Erklärung nicht erteilt wird.

Mannheim, den 02.04.2024 Unterschrift

F. Krömer